

SOC Home Lab / Phase 1

Author: Andy Hernandez

Objective: The objective of this project is to build an isolated, multi-zone SOC homelab to practice Purple Team operations. The environment covers defensive (Blue Team) capabilities—including firewall engineering, SIEM logging (Wazuh), and EDR telemetry—alongside offensive (Red Team) tactics such as reconnaissance, vulnerability exploitation, and adversary simulation to build and refine custom detection rules.

Assets & Roles

- **OPNsense Firewall & Gateway:** An open-source stateful firewall deployed as the central routing engine. Manages traffic flow across virtual network adapters, enforcing strict access control lists (ACLs) between the WAN, Corporate LAN, and SOC Management zones.
- **RHEL 10 (Wazuh SIEM Host):** A RHEL virtual machine hosting the Wazuh Security Information and Event Management (SIEM). Uses standard RHEL repositories (BaseOS and AppStream) along with the official Wazuh repository to support log ingestion, correlation, and alert generation.
- **Windows Server 2019 (Domain Controller):** Serves as the central directory server managing identity, group policies, and authentication services (Active Directory / DNS) for the internal enterprise domain.
- **Windows 10 Workstation:** An Active Directory domain-joined endpoint configured to replicate a standard corporate client machine.
- **Kali Linux Virtual Machine:** This virtual machine will be used to conduct vulnerability scanning and practicing certain attack vectors on the Windows Server and Workstation.

Virtual environment setup: The first goal of this project is to set up two virtual network adapters:

New virtual network adapter one – This will be the LAN that will serve as the gateway to the OPNsense Firewall for the Windows Server and Windows 10 Workstation.

New virtual network adapter two – This will serve as the second LAN or SecurityManagement Interface as it will be called throughout this document. It will serve as the gateway to the OPNsense Firewall for the RHEL virtual machine.

Virtual network adapter setup: I first began by starting up the hypervisor application VMware Workstation Pro. I proceeded to the virtual network editor where I identified all virtual network adapters:

- **VMnet1 (Host-only)**
- **VMnet8 (NAT)**

I proceeded to add a new virtual network adapter, VMnet2. This will serve as the LAN interface for the firewall and all hosts will receive their IP addresses via DHCP from the firewall. The firewall being used will be OPNsense Version 26.7.

The next virtual network adapter being added is VMnet3. This virtual network adapter will serve as a SecurityManagement Interface where the RHEL virtual machine will reside. The purpose of separating these two interfaces is to simulate a real enterprise environment. If an attacker were to somehow gain access to the Windows Workstation on VMnet2, lateral movement to VMnet3 would be difficult or impossible to achieve since communication between the two interfaces is limited.

Firewall setup: I proceeded to download the DVD ISO from OPNsense official website. Once done I configured the settings for the VM such as processors, cores, RAM, disk space, and virtual network adapters being used.

I was then presented with a live environment where I could configure the firewall's settings. I will be assigning the WAN and LAN IP addresses.

```
login: root
Password:
-----
:      Hello, this is OPNsense 26.7      :
:                                       :
: Website:   https://opnsense.org/      :
: Handbook:  https://docs.opnsense.org/ :
: Forums:    https://forum.opnsense.org/ :
: Code:      https://github.com/opnsense :
: Reddit:    https://reddit.com/r/opnsense :
:                                       :
-----

*** OPNsense.internal: OPNsense 26.7 (amd64) ***

      No network interfaces are assigned.

0) Logout                               7) Ping host
1) Assign interfaces                     8) Shell
2) Set interface IP address              9) pfTop
3) Reset the root password               10) Firewall log
4) Reset to factory defaults             11) Reload all services
5) Power off system                      12) Update from console
6) Reboot system                         13) Restore a backup

Enter an option: █
```

I proceeded with option one to assign the interface names where:

WAN → em0

LAN → em1

SecurityManagement → em2

```
The interfaces will be assigned as follows:
```

```
WAN  -> em0
```

```
LAN  -> em1
```

```
Do you want to proceed? [y/N]: █
```

*Note: I added the SecurityManagement interface later and did not capture the process. I forgot to add the adapter in the VMware Workstation settings prior to booting up the OPNsense Firewall.

I then continued the configuration process and assigned IP address to the interfaces where:

WAN → 192.168.116.148/24

LAN → 192.168.50.2/24

It's important to note that the WAN interface obtained its IPv4 address via DHCP through VMware Workstation. I disabled DHCP for VMnet2 and VMnet3 as they will be obtaining their IPv4 addresses via the OPNsense Firewall. The LAN and SecurityManagement interfaces I will be manually adding static IPv4 addresses.

I then rebooted the virtual machine to verify if the IP address changes to em0, em1, and em2 would be persistent after rebooting. After further review all interfaces were changed back to their default IP address and were not staying consistent.

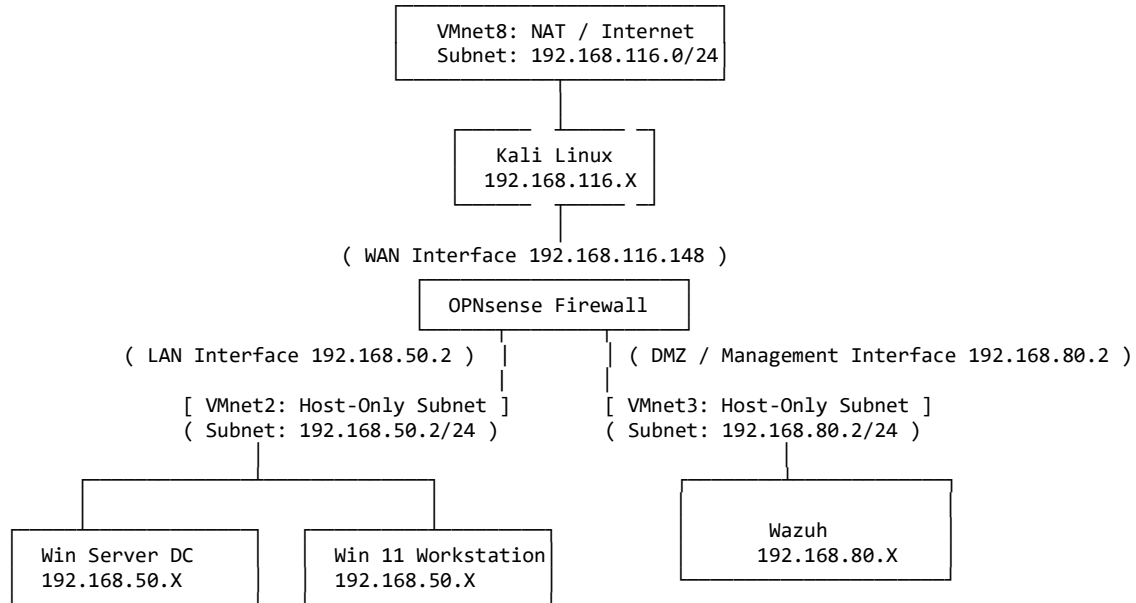
After further investigation it was evident the virtual machine was running entirely off temporary memory using the live iso. I solved this issue by installing the virtual machine on the virtual disk instead of booting from the CD/DVD.

After powering off the virtual machine and adjusting the settings regarding the CD/DVD (IDE), I powered on the virtual machine to validate my changes persisted.

I then made my way to the web-based GUI validate my changes at <http://192.168.X.X/index.php> and was successful. It is now time to begin firewall configurations.

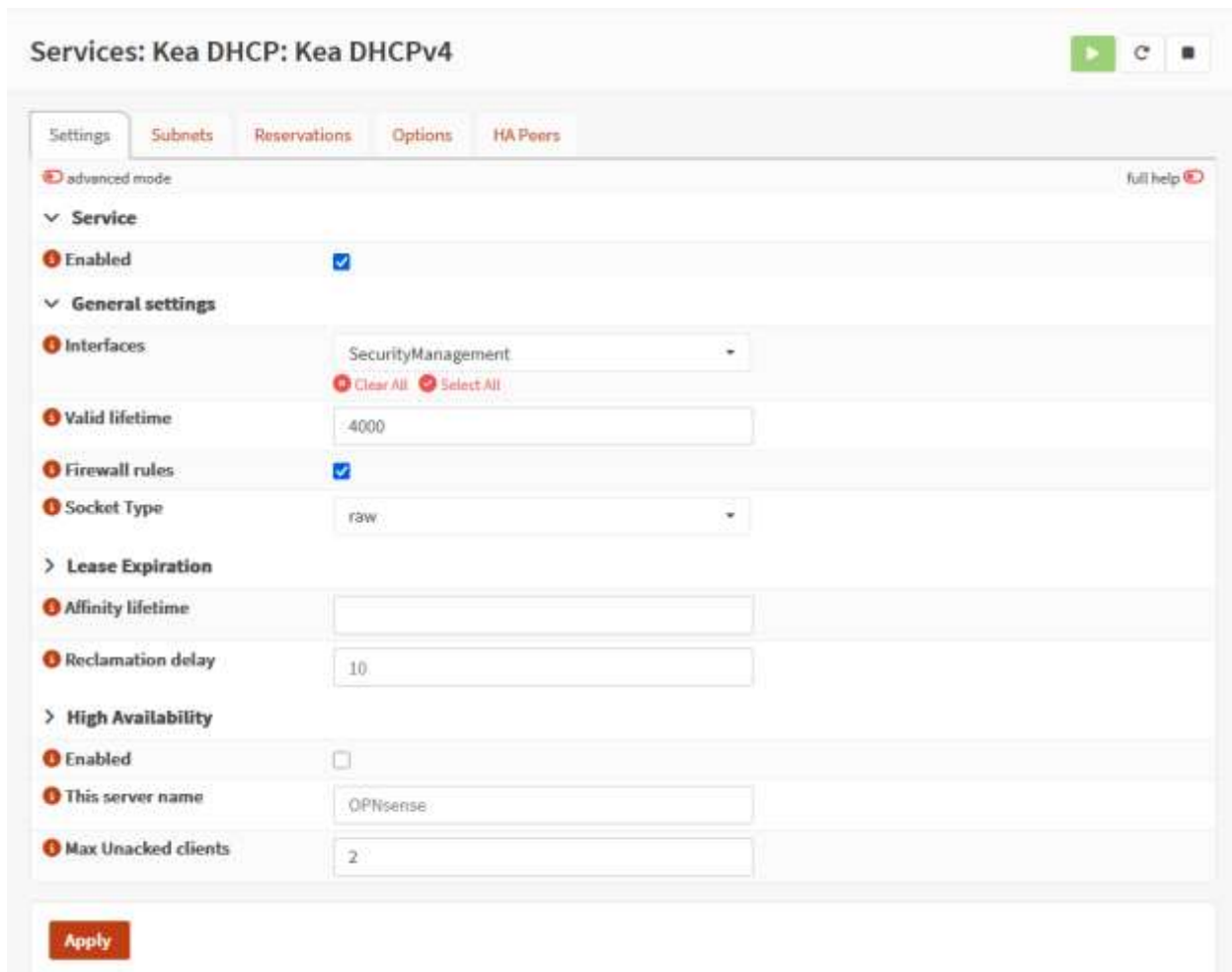
OPNsense Configurations: After successfully adding static IPv4 addresses to the WAN, LAN, and SecurityManagement interfaces I began configuring the firewall settings via the startup wizard. I established a domain name and unblocked private networks on all interfaces. OPNsense by default blocks traffic from private IP addresses. The Kali virtual machine that will be used in this lab to simulate attacks will reside on virtual network adapter VMnet8. This will help in simulating real traffic. The network architecture so far can be seen below.

Network Architecture:



- Disabling dnsmasq DHCP service binding to port 67
- Assigning a pool of IP address for the LAN / Security Management interfaces

Kea DHCP: I then proceeded with the next configuration in the OPNsense Firewall web UI, DHCP. I proceeded by enabling Kea DHCP for IPv4 addresses as hosts added to the LAN or SecurityManagement subnet will obtain their IPv4 address through this service offered by the firewall.



The screenshot shows the 'Services: Kea DHCP: Kea DHCPv4' configuration page in the OPNsense web UI. The page has tabs for 'Settings', 'Subnets', 'Reservations', 'Options', and 'HA Peers'. The 'Settings' tab is active, showing a configuration form. At the top right, there are icons for play, refresh, and a menu. Below the tabs, there's a 'full help' link. The form is organized into sections: 'Service' (with an 'Enabled' checkbox checked), 'General settings' (with 'Interfaces' set to 'SecurityManagement', 'Valid lifetime' set to '4000', 'Firewall rules' checked, and 'Socket Type' set to 'raw'), 'Lease Expiration' (with 'Affinity lifetime' and 'Reclamation delay' fields), and 'High Availability' (with 'Enabled' unchecked, 'This server name' set to 'OPNsense', and 'Max Unacked clients' set to '2'). An 'Apply' button is at the bottom left.

I then started up my RHEL VM to see if an IP address was assigned.

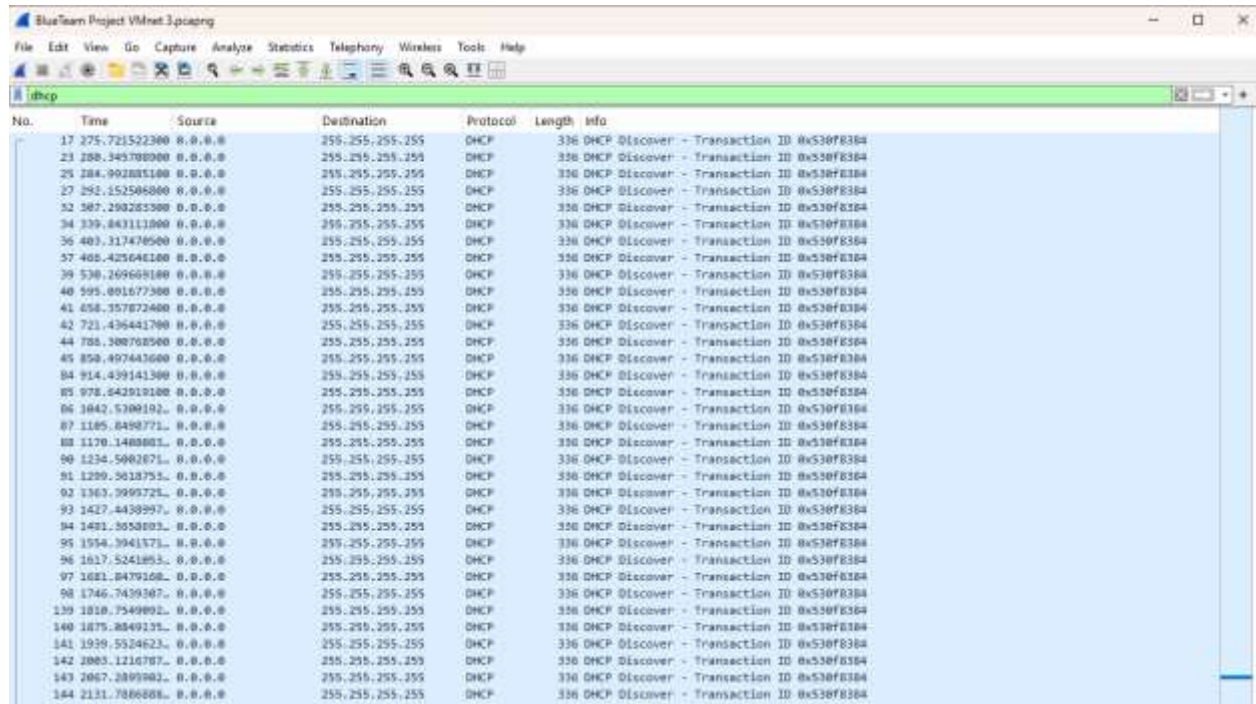
After running the command “*ip a*” on the RHEL VM, I learned there was no IP address assigned to my virtual machine. I began researching possible issues that could be the reason my RHEL VM was not able to receive an IP address. I then review how DHCP works.

DHCP works as follows:

- **Discover** – The service discovers a client broadcasting the request for an IP address. Usually from the source IP address 0.0.0.0 with a broadcast / Destination address of 255.255.255.255
- **Offer** – The DHCP service on the interface will send an offer for an IP address to the client.

- **Request** – The client will request a lease for the IP address being offered.
- **Acknowledge** – The server will acknowledge the request and assigned the client with the new IP address.

My first course of action was to open Wireshark and see if the client was sending out a request for an IP address.



The image shows a Wireshark packet capture window titled "BlueTeam Project VMnet3.pcapng". The filter is set to "dhcp". The packet list shows 184 DHCP Discover packets. Each packet is from a different source IP (e.g., 17 275.721522300, 23 288.345788000, etc.) and is destined to 255.255.255.255. The protocol is DHCP and the length is 336 bytes. The info column for each packet shows "336 DHCP Discover - Transaction ID 0x530f8364".

No.	Time	Source	Destination	Protocol	Length	Info
17	275.721522300	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
23	288.345788000	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
25	288.992885100	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
27	292.152508800	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
32	307.290285300	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
34	339.843113000	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
36	403.317470500	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
37	405.425640100	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
39	530.269668100	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
40	595.001677500	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
41	658.357872400	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
42	721.436441700	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
44	788.308705500	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
45	850.497443600	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
84	914.439141300	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
85	978.642919100	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
86	1042.5706192	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
87	1105.6498771	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
88	1170.1408801	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
90	1234.5002871	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
91	1299.5018751	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
92	1363.9999721	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
93	1427.4438997	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
94	1491.3058803	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
95	1554.3941371	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
96	1617.5241893	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
97	1681.0479168	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
98	1746.7439367	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
139	1818.7549892	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
140	1875.8849131	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
141	1939.5524623	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
142	2003.1216787	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
143	2067.2899982	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364
184	2131.7886888	8.0.0.0	255.255.255.255	DHCP	336	DHCP Discover - Transaction ID 0x530f8364

I confirmed that the client or RHEL VM was sending out a request for an IP address and confirmed that was working. I now knew this was an issue regarding my firewall or the DHCP service.

After more research I noticed that I did not have a firewall rule in place to allow traffic from my RHEL VM to the OPNsense firewall, so I created a new firewall rule to allow traffic on the SecurityManagement interface as follows:

Interface	Version	Direction	Protocol	Source	Port	Dest	Port	Action
SecurityManagement	IPv4	IN	UDP	*	68	*	67	Allow

I went with the following because DHCP runs on ports 67 and 68. Clients listen to all incoming OFFERS and ACKS from DHCP servers on port 68 and I set the destination to be any server on port 67. Port 67 is where the DHCP server listens to DISCOVERS and REQUESTS from clients.

Now that the firewall was set up to send and receive traffic from VMnet3 for DHCP, it was time to test out if the RHEL VM would receive an IPv4 address. Unfortunately, the VM did not receive an IPv4 address.

After further research it was evident that I did not configure the DHCP settings as required for the service to work properly so I did the following:

- Assign a subnet range for VMnet3
- Assign a pool of available IP addresses for hosts requesting an IP address.

I began researching more and decided to move towards reading the logs for Kea DHCP and discovered the following:

2026-07-25T21:55:18	Warning	kea-dhcp4	WARN [kea-dhcp4.dhcp4srv.0x4d16e0669010] DHCP4SRV_OPEN_SOCKET_FAIL failed to open socket: Failed to open socket on interface em2, reason: failed to bind fallback socket to address 192.168.80.2, port 67, reason: Address already in use - is another DHCP server running?
---------------------	---------	-----------	--

It appears that the Kea DHCP v4 service failed to open the socket on interface em2 (SecurityManagement). The reason was the service failed to bind to the socket or, in other words port 67. After further investigating it was revealed that dnsmasq was binding to port 67 and not allowing Kea DHCP service to communicate.

I used the command “**sockstat -4 -l -p 67**” in the terminal for the OPNsense firewall.

- Sockstat prints all sockets and the details associated with them such as IP, protocol, command, user, and PID.
- -4 flag lists only IPv4 addresses
- -l lists only sockets listening
- -p specifies the port, in this case 67 to identify the DHCP server listening for broadcasts from clients.

After validating dnsmasq was the culprit behind this issue I disabled the dnsmasq DHCP service within the Web GUI of OPNsense and restarted the Kea DHCP service. I checked back in the terminal to verify Kea was binding to the socket.

```
root@OPNsense:~ # sockstat -4 -l -p 67
USER COMMAND      PID FD PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
root kea-dhcp4    28345 18 udp4   192.168.80.2:67  *:*
```

The RHEL VM now successfully obtained an IPV4 address associated with SecurityManagement interface / pool range. It was now time to test if I could obtain the Wazuh services needed to deploy a Wazuh Web UI.

RHEL 10 VM configurations / troubleshooting: I first began by signing up for a developers account with RedHat as it gives access to RedHat's repositories such as BaseOS and AppStream. I then downloaded a RHEL 10 DVD ISO image and started a virtual instance on VMware Workstation.

I was then prompted to register my RHEL VM with the subscription manager to obtain access to the repositories mentioned above. Unfortunately, it was not as simple as I hoped it would be. I proceeded to run the command "*subscription-manager register*" to register my virtual instance, but I received the error: "*Cannot resolve hostname*". It immediately came to my attention that my RHEL VM could not resolve hostnames. I began trying to discover the root of the issue and my only lead was that it was related to DNS.

I immediately opened Wireshark to begin a packet capture and see what was going on in better detail:

1	0.000000000	VMware_	Broadcast	ARP	42	Who has 192.168.50.2? Tell 192.168.80.10
2	1.015805000	VMware_	Broadcast	ARP	42	Who has 192.168.50.2? Tell 192.168.80.10
3	2.039802500	VMware_	Broadcast	ARP	42	Who has 192.168.50.2? Tell 192.168.80.10

I noticed that my RHEL VM was sending out an ARP broadcast to identify what the MAC address was for my VMnet2 virtual network adapter or in other words the LAN interface for the OPNsense firewall.

I then tried to visualize the flow of traffic and where it would begin and end. The flow of traffic should be:

RHEL VM → Virtual Switch: VMnet3 → OPNsense: VMnet3 Interface → OPNsense VMnet8 WAN Interface → VMware NAT Engine / Host PC → Internet

After understanding the flow of traffic, I understood that my RHEL VM should not be looking for an IP address related to VMnet2, it should be looking for OPNsense VMnet3 Interface. I researched how to resolve this issue and learned the nameserver filed in the /etc/resolv.conf needed to be modified. The resolv.conf file is a configuration file for the C library resolver in Unix-like operating systems. It tells your operating system how and where to translate human-readable domain names (like google.com).

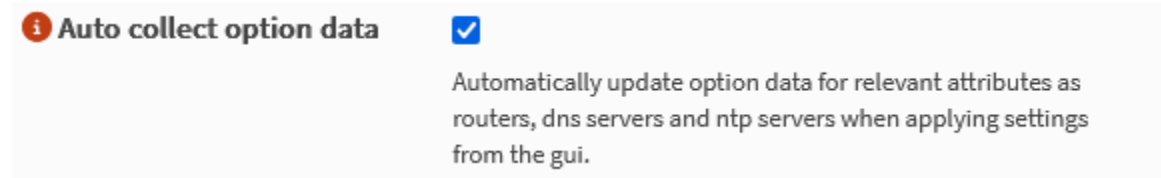
Based on my initial configuration in the web UI of OPNsense I misconfigured the Kea DHCP to point to VMnet2 interface when attempting to make an outbound connection. I corrected the file to:

```
root@localhost:~# cat /etc/resolv.conf

nameserver 192.168.80.2
```

There should be a field also labeled "*search*" following with the domain name, but I am leaving that portion out of the snapshot up above.

I also modified the Kea DHCP service through the web UI to as follows:



Enabling this setting should ensure that the changes I made to the /etc/resolv.conf stay persistent after reboot.

I now decided to validate if my if my changes solved the issue at hand which was registering my RHEL VM to gain access to the repositories needed. Unfortunately, the issue persisted.

I began researching some more to identify what the culprit could possibly be, and I found what my next culprit was. I did not have a firewall rule set to allow traffic from the SecurityManagement interface (Vmnet3) to the WAN. OPNsense by default only creates one rule for the first added LAN interface to allow traffic from LAN to WAN. The first added LAN interface was for VMnet2 so there was no firewall rule enabling traffic to the WAN from VMnet3.

I proceeded to create a rule to allow traffic to enter via VMnet3 if the source IPv4 address was from the RHEL VM.

Interface	Version	Direction	Protocol	Source	Port	Dest	Port	Gateway	Action
SecurityManagement	IPv4	IN	*	192.168.80.10	*	*	*	*	Allow

This rule created should allow the RHEL VM on VMnet3 internet access.

I looked to validate my configurations and pinged google from my RHEL VM.

I first did “**ping 8.8.8.8**” to see if an internet connection was being made and it was successful. I was able to send and receive ICMP packets.

→	664	3000.0056565	192.168.80.10	8.8.8.8	ICMP	98 Echo (ping) request	id=0x0002, seq=2/512, ttl=64 (reply in 665)
←	665	3000.0199692	8.8.8.8	192.168.80.10	ICMP	98 Echo (ping) reply	id=0x0002, seq=2/512, ttl=127 (request in 664)
→	666	3001.0072002	192.168.80.10	8.8.8.8	ICMP	98 Echo (ping) request	id=0x0002, seq=3/768, ttl=64 (reply in 667)
←	667	3001.0207627	8.8.8.8	192.168.80.10	ICMP	98 Echo (ping) reply	id=0x0002, seq=3/768, ttl=127 (request in 666)

I then looked to see if my RHEL VM could resolve a hostname. I ran the command “**ping google.com**”. Sadly, the issue persisted.

I am moving forward in the right direction as I can reach hosts on the outside from my RHEL VM which is behind my OPNsense firewall on VMnet3. I proceeded to research again on what my next issue could possibly be. I opened Wireshark again and performed a packet capture on VMnet3 and I ran the command “**ping google.com**” from my RHEL VM to see what was happening:

668	3008.0320875	192.168.80.10	192.168.80.2	DNS	70 Standard query 0x5435 A google.com
669	3008.0321132	192.168.80.10	192.168.80.2	DNS	70 Standard query 0x4234 AAAA google.com
670	3008.0334466	192.168.80.2	192.168.80.10	DNS	70 Standard query response 0x5435 Server failure A google.com
671	3008.0334911	192.168.80.2	192.168.80.10	DNS	70 Standard query response 0x4234 Server failure AAAA google.com

I saw in the query response there was a server failure, and I began to look further into that response to see what was causing it.

▼ Flags: 0x8182 Standard query response, Server failure

```

1... .. = Response: Message is a response
.000 0... .. = Opcode: Standard query (0)
.... 0... .. = Authoritative: Server is not an authority for domain
.... 0... .. = Truncated: Message is not truncated
.... 1... .. = Recursion desired: Do query recursively
.... 1... .. = Recursion available: Server can do recursive queries
.... 0... .. = Z: reserved (0)
.... 0... .. = Answer authenticated: Answer/authority portion was not authenticated by the server
.... 0... .. = Non-authenticated data: Unacceptable
.... 0010 = Reply code: Server failure (2)

```

It appears that DNSSEC failed. The server was unable to validate the cryptographic signature (or DNSSEC validation failed) for the domain record. I decided to research as to why the authentication failed. The main reasons I discovered were:

- Firewall rules on SecurityManagement interface not allowing DNS queries
- DNSSEC requires precise time to validate cryptographic signatures, time drifts cause Unbound DNS query to fail and not authenticate.

I learned through the firewall logs that the SecurityManagement interface was denying traffic on port 123 (Network Time Protocol) and port 53 (DNS).

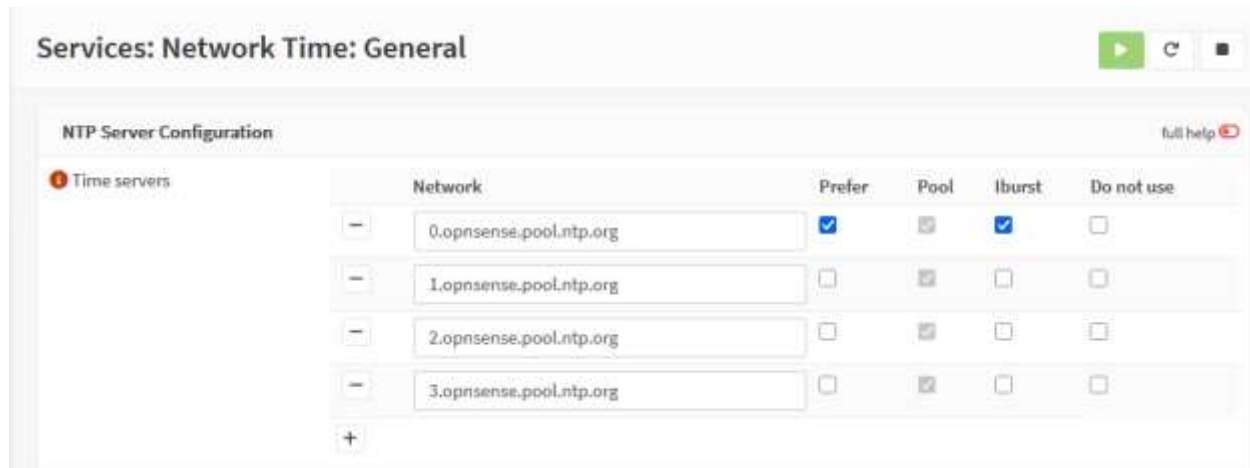
Security...	In	2026-07-28T23:35:20	UDP	192.168...	192.168.80.2:53	block	Default deny / state violation rule
Security...	In	2026-07-28T23:35:15	UDP	192.168...	192.168...	block	Default deny / state violation rule
Security...	In	2026-07-28T23:08:22	UDP	192.168...	192.168.80.2:123	block	Default deny / state violation rule
Security...	In	2026-07-28T23:08:20	UDP	192.168...	192.168...	block	Default deny / state violation rule

My first course of action was to create firewall rules to allow traffic on ports 123 and 53 for the SecurityManagement Interface and restart the Network Time service.

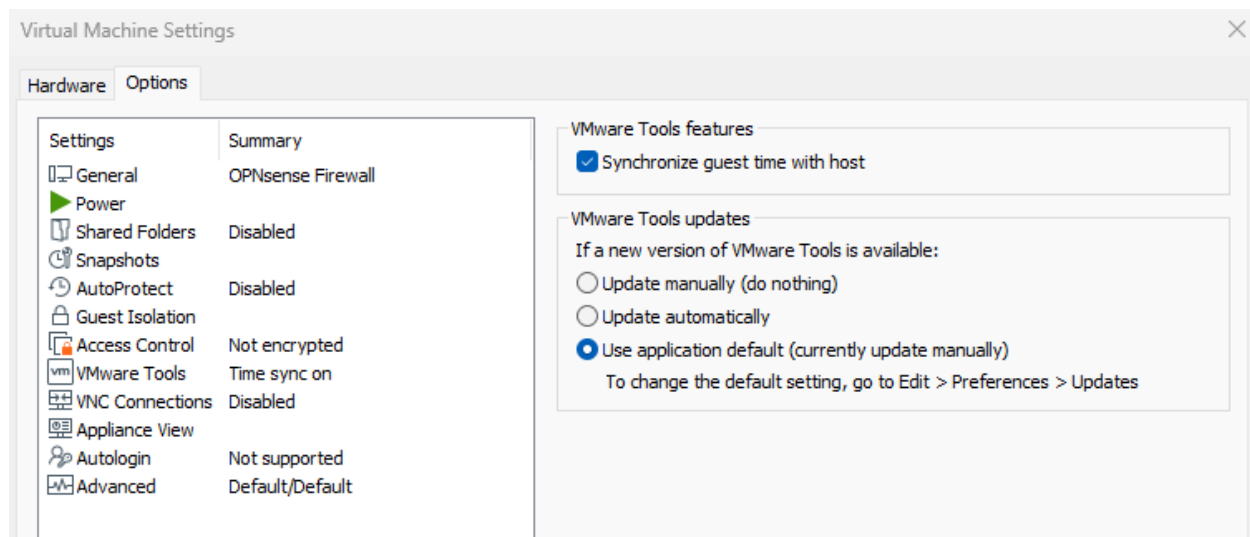
I first created the new rules:

	Version	Protocol	Source	Port	Destination	Port	Gateway	Description	
☐	SecurityMa nagement	IPv4	UDP	192.168.80.10	*	192.168.80.2	53	*	SecurityMana gement DNS
☐	SecurityMa nagement	IPv4	UDP	192.168.80.10	*	192.168.80.2	123	*	SecurityMana gement NTP

I then restarted the Network Time service and configured the settings in VMware Workstation to ensure the time change persisted.



This option in VMware Workstation enables time synchronization with the host PC to ensure the time is correct upon reboot.



I also configured DNS forwarding on the OPNsense firewall to DNS server 8.8.8.8 and 1.1.1.1 (google and cloudflare).



I then rebooted the firewall and RHEL VM and validated my changes by verifying the times were correct on the OPNsense dashboard as well as attempting to ping google.com.

```
wazuh-user@localhost:~$ ping google.com
PING google.com (172.217.165.206) 56(84) bytes of data.
64 bytes from lax31s06-in-f14.1e100.net (172.217.165.206): icmp_seq=1
  ttl=127 time=13.0 ms
64 bytes from lax31s06-in-f14.1e100.net (172.217.165.206): icmp_seq=2
  ttl=127 time=13.4 ms
64 bytes from lax31s06-in-f14.1e100.net (172.217.165.206): icmp_seq=3
  ttl=127 time=14.4 ms
64 bytes from lax31s06-in-f14.1e100.net (172.217.165.206): icmp_seq=4
  ttl=127 time=13.9 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3103ms
rtt min/avg/max/mdev = 12.959/13.670/14.391/0.537 ms
wazuh-user@localhost:~$
```

I was able to successfully resolve a hostname and then I proceeded to register my RHEL VM with the RedHat's subscription manager and verify I had access to the BaseOS and AppStream repositories via "***dnf repolist***" and "***dnf update -y***"

```
wazuh-user@localhost:~$ dnf repolist
Not root, Subscription Management repositories not updated
repo id                                repo name
rhel-10-for-x86_64-appstream-rpms      Red Hat Enterprise Linux 10 for x86_64 - AppStream (RPMs)
rhel-10-for-x86_64-baseos-rpms         Red Hat Enterprise Linux 10 for x86_64 - BaseOS (RPMs)
wazuh-user@localhost:~$
```

The next step is to install Wazuh on the RHEL VM. Wazuh requires these three services to work together to successfully perform its tasks:

- **Wazuh Server** – Collects raw log data sent from agents and reads through all the logs comparing them against rules to see if they are suspicious.
- **Wazuh Indexer** – The indexer stores the alerts collected by the server and organizes the logs so the user can search through them efficiently.
- **Wazuh Dashboard** – The web GUI used to visualize and manage the logs stored by the indexer.

I ran this command to download the automated script provided by Wazuh to install everything needed:

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

The curl command will download the needed script “wazuh-install.sh” in my current working directory: */home/wazuh-user* which I will run inside my RHEL VM to make the installation process easy.

```
wazuh-user@localhost:~$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

The next step is to run the script, but execution permission needs to be added to the script. I ran the command “*sudo chmod 744 wazuh-install.sh*” this will assign the owner of the file with full permission and read access to the group assigned to the script and world.

```
wazuh-user@localhost:~$ ls -la | grep wazuh-install.sh
-rwxr--r--. 1 wazuh-user wazuh-user 208288 Jul 29 17:25 wazuh-install.sh
wazuh-user@localhost:~$
```

The script was now ready to be executed. I ran the command “*sudo ./wazuh-installer.sh -a*”. The -a options install and configure Wazuh server, Wazuh indexer, Wazuh dashboard.

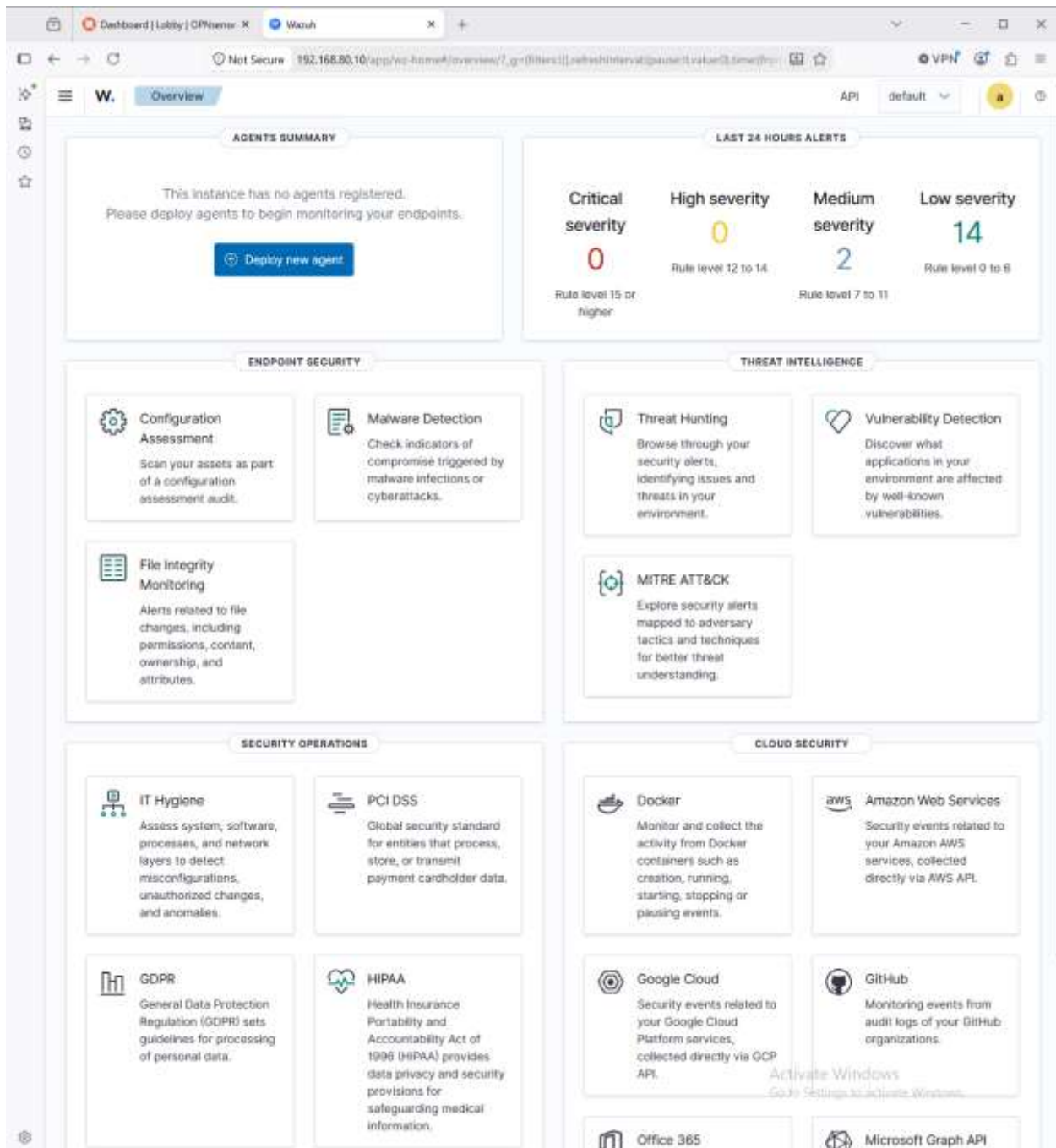
```

wazuh-console@root:~$ sudo ./wazuh-install.sh -s
29/07/2026 18:04:15 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.7
29/07/2026 18:04:15 INFO: Verbose logging redirected to /var/log/wazuh-install.log
29/07/2026 18:04:15 INFO: The recommended systems are: Red Hat Enterprise Linux 7, 8, 9; CentOS 7, 8; Amazon Linux 2; Amazon Linux 2023; Ubuntu 16.04, 18.04, 20.04, 22.04; Rocky Linux 8, 9.
29/07/2026 18:04:15 WARNING: The current system does not match with the list of recommended systems. The installation may not work properly.
29/07/2026 18:04:15 INFO: Verifying that your system meets the recommended minimum hardware requirements.
29/07/2026 18:04:15 INFO: Wazuh web interface port will be 443.
29/07/2026 18:04:15 WARNING: The system has FirewallD enabled. Please ensure that traffic is allowed on these ports: 1515, 1514, 443.
29/07/2026 18:04:16 INFO: Wazuh repository added.
29/07/2026 18:04:16 INFO: --- Configuration files ---
29/07/2026 18:04:16 INFO: Generating configuration files.
29/07/2026 18:04:16 INFO: Generating the root certificate.
29/07/2026 18:04:16 INFO: Generating Admin certificates.
29/07/2026 18:04:16 INFO: Generating Wazuh indexer certificates.
29/07/2026 18:04:16 INFO: Generating Filebeat certificates.
29/07/2026 18:04:16 INFO: Generating Wazuh dashboard certificates.
29/07/2026 18:04:16 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
29/07/2026 18:04:16 INFO: --- Wazuh indexer ---
29/07/2026 18:04:16 INFO: Starting Wazuh indexer installation.
29/07/2026 18:05:36 INFO: Wazuh indexer installation finished.
29/07/2026 18:05:36 INFO: Wazuh indexer post-install configuration finished.
29/07/2026 18:05:36 INFO: Starting service wazuh-indexer.
29/07/2026 18:05:43 INFO: wazuh-indexer service started.
29/07/2026 18:05:43 INFO: Initializing Wazuh indexer cluster security settings.
29/07/2026 18:05:44 INFO: Wazuh indexer cluster security configuration initialized.
29/07/2026 18:05:44 INFO: Wazuh indexer cluster initialized.
29/07/2026 18:05:44 INFO: --- Wazuh server ---
29/07/2026 18:05:44 INFO: Starting the Wazuh manager installation.
29/07/2026 18:06:32 INFO: Wazuh manager installation finished.
29/07/2026 18:06:32 INFO: Wazuh manager vulnerability detection configuration finished.
29/07/2026 18:06:32 INFO: Starting service wazuh-manager.
29/07/2026 18:06:46 INFO: wazuh-manager service started.
29/07/2026 18:06:46 INFO: Starting Filebeat installation.
29/07/2026 18:06:56 INFO: Filebeat installation finished.
29/07/2026 18:06:57 INFO: Filebeat post-install configuration finished.
29/07/2026 18:06:57 INFO: Starting service filebeat.
29/07/2026 18:06:57 INFO: Filebeat service started.
29/07/2026 18:06:57 INFO: --- Wazuh dashboard ---
29/07/2026 18:06:57 INFO: Starting Wazuh dashboard installation.
29/07/2026 18:11:12 INFO: Wazuh dashboard installation finished.
29/07/2026 18:11:12 INFO: Wazuh dashboard post-install configuration finished.
29/07/2026 18:11:12 INFO: Starting service wazuh-dashboard.
29/07/2026 18:11:12 INFO: wazuh-dashboard service started.
29/07/2026 18:11:13 INFO: Updating the internal users.
29/07/2026 18:11:14 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.
29/07/2026 18:11:22 INFO: The filebeat.yml file has been updated to use the Filebeat KeyStore username and password.
29/07/2026 18:11:54 INFO: Initializing Wazuh dashboard web application.
29/07/2026 18:11:54 INFO: Wazuh dashboard web application initialized.
29/07/2026 18:11:54 INFO: --- Summary ---
29/07/2026 18:11:54 INFO: You can access the web interface https://(wazuh-dashboard-ip):443

```

The end of the script provides a username and password to access the web UI via <https://<wazuh-dashboard-ip>:443>

The next step was to see if the Wazuh VM was working properly, I needed to validate if I had access to the Web GUI.

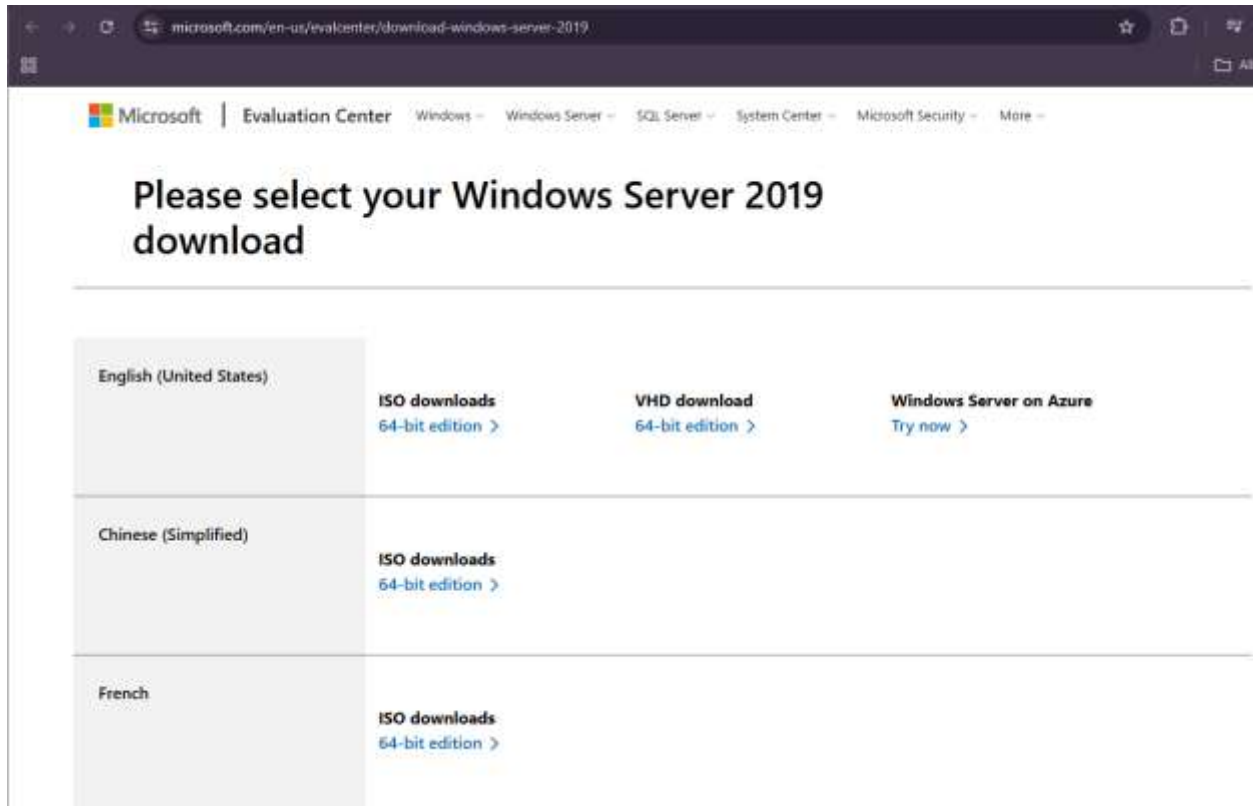


RHEL VM was now successfully set up and ready to deploy agents to begin collecting telemetry from the Windows Server and Windows Workstation. The next portion of phase one will now focus on setting up and deploying the Windows Server and Windows Workstation. I will be following the same process where I will enable DHCP for the LAN interface, assign a subnet and pool range for available IP addresses, and finally create a firewall rule to ensure communication between the hosts in LAN and the OPNsense firewall.

Windows Server deployment: The first thing is to secure a Windows Server ISO. I proceeded to Microsoft's website and downloaded both the Windows Server ISO and well and the media creation tool to obtain a Windows 10 ISO.

I went to this website to first obtain the Windows Server 2019 DVD ISO:

<https://www.microsoft.com/en-us/evalcenter/download-windows-server-2019>



I proceeded to download the English 64-bit edition and began searching for the Windows 10 ISO. Obtaining a Windows 10 ISO was slightly different as I needed to download a media creation tool. I first went to this website here:

<https://www.microsoft.com/en-us/software-download/windows10>

microsoft.com/en-us/software-download/windows10

Microsoft | Software Download Windows Windows Insider Preview All Microsoft ME

Download Windows 10

Before updating, please refer to the [Windows release information status](#) for known issues to confirm your device is not impacted.

Stay Secure with Essential Windows Updates: After October 14, 2025, Windows 10 will no longer receive free software updates, technical support, or security fixes. This means your PC will be more vulnerable to security threats and malware. **Consider Upgrading to Windows 11:** Move to the security, speed, and innovation that Windows 11 PCs provide, available at every price point. Upgrading to Windows 11 ensures you continue to receive the latest security updates, features, and technical support, keeping your PC safe and efficient. For more information on preparing for Windows 10 end of support, see our [Windows blog post](#).

Windows 10 2022 Update | Version 22H2

The Update Assistant can help you update to the latest version of Windows 10. To get started, click **Update now**.

[Update now](#)

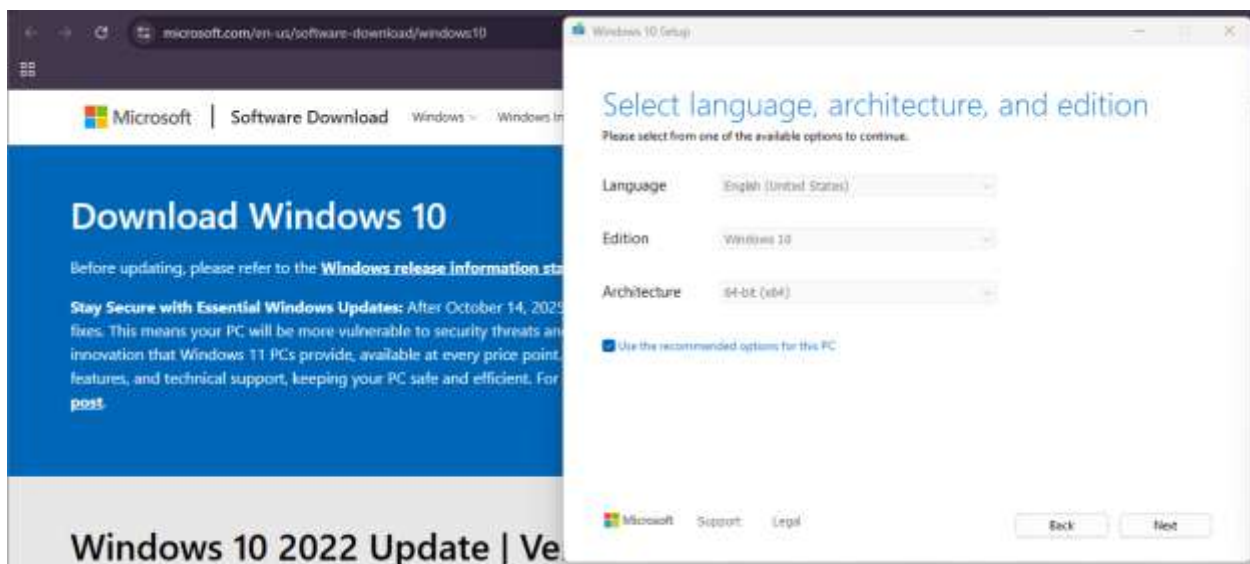
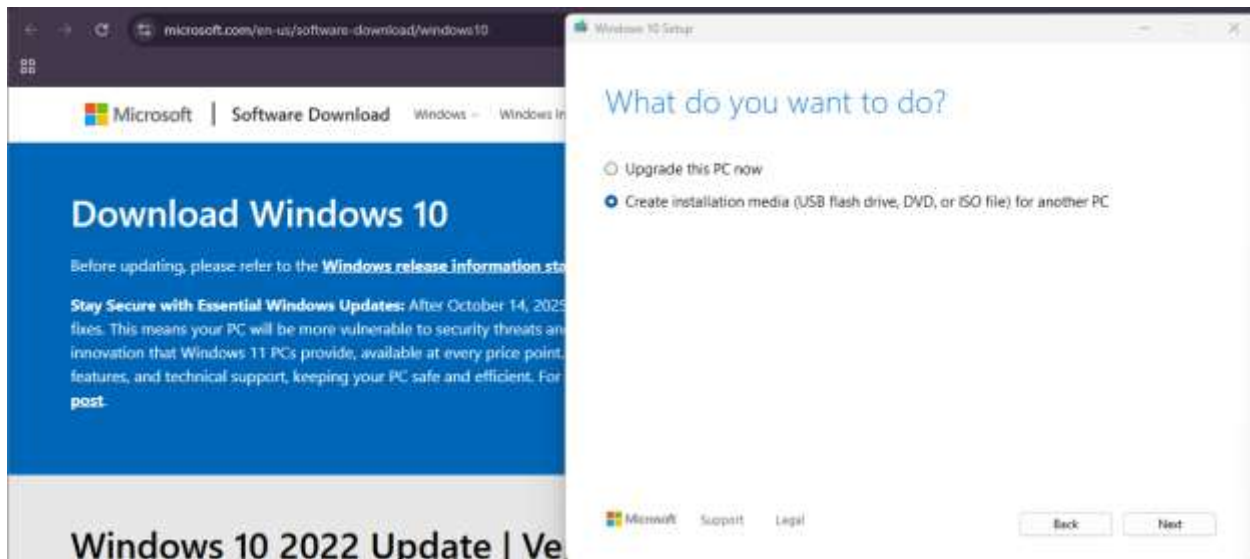
Create Windows 10 installation media

To get started, you will first need to have a license to install Windows 10. You can then download and run the media creation tool. For more information on how to use the tool, see the instructions below.

[Download Now](#)



I then proceeded to execute the media creation tool that I downloaded

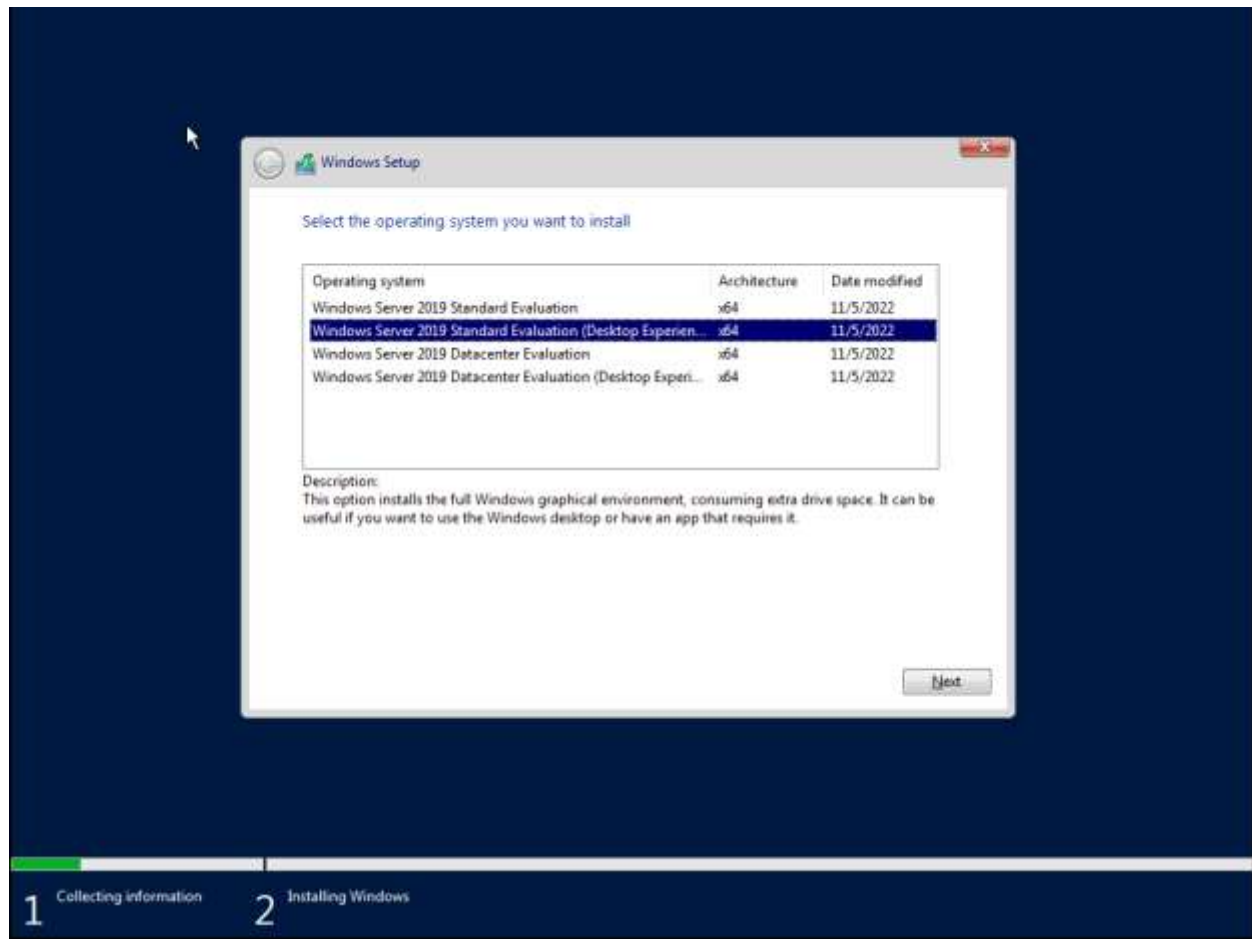


After completing the rest of the steps provided by the media creation tool, I finally obtained a Windows 10 ISO.

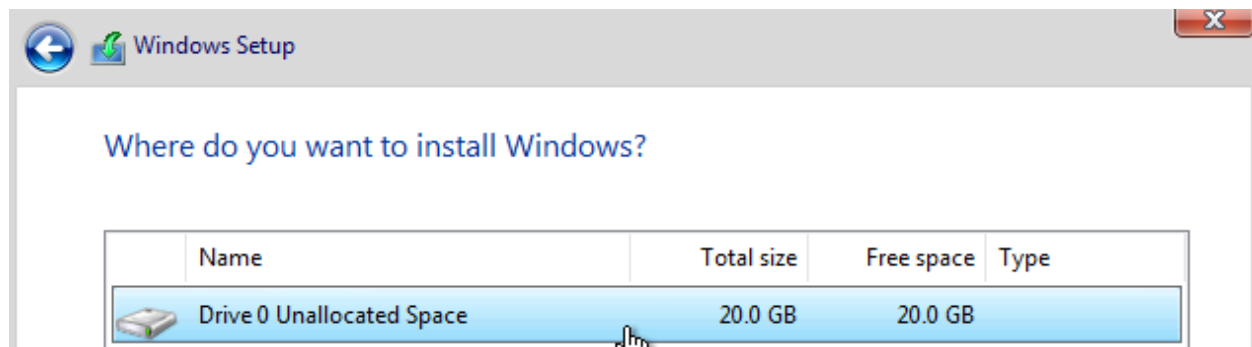
 Windows 10 Workstation	7/29/2026 9:42 PM	Disc Image File	4,779,200 KB
--	-------------------	-----------------	--------------

It was now time to deploy both virtual machines and configure them properly in my lab environment.

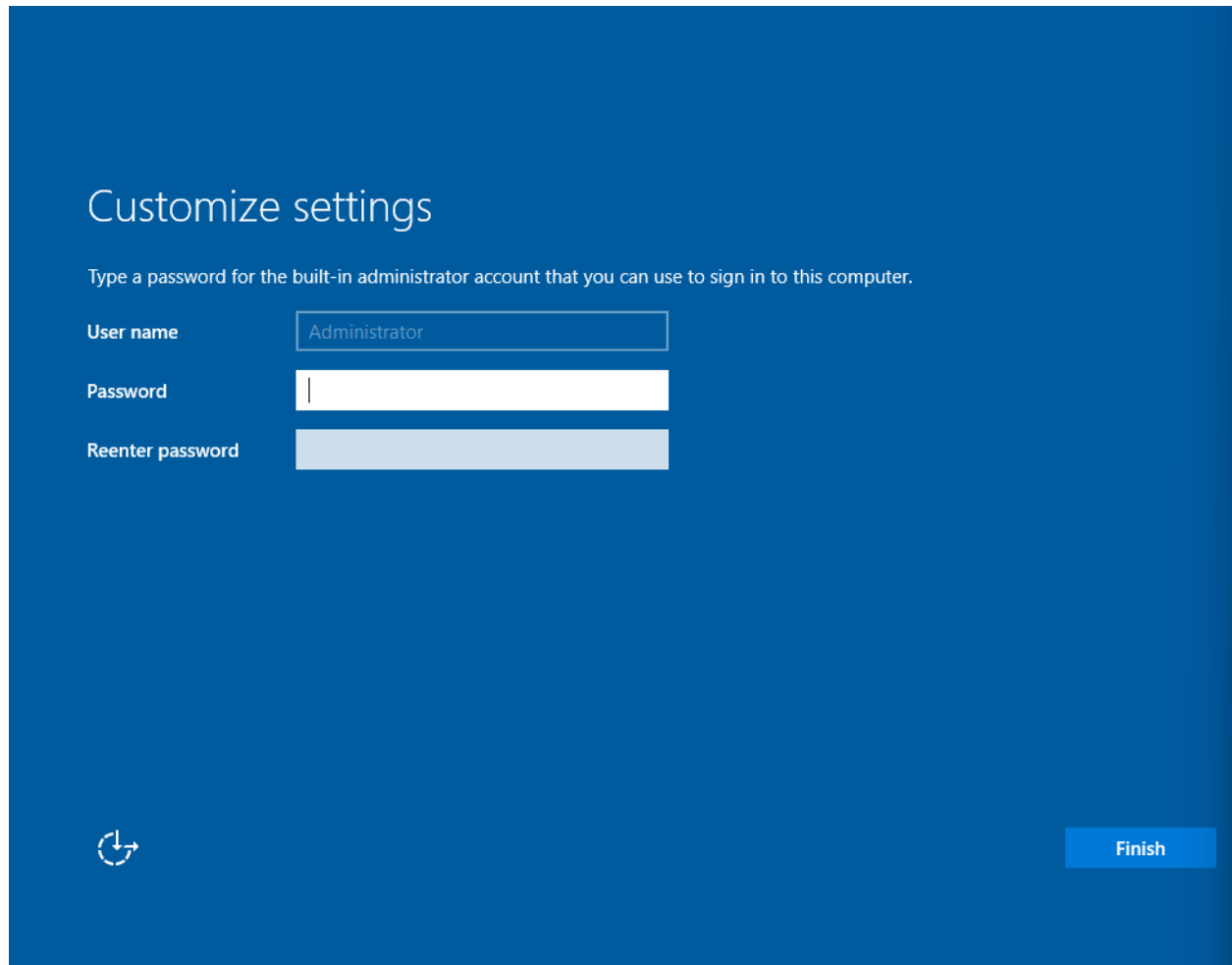
I proceeded to allocate resources to my Windows Server and proceeded to boot my virtual machine from a CD/DVD using the ISO image and followed the installation guide.



I went with the Windows Server 2019 Standard Evaluation (Desktop Experience) as I wanted a GUI to interact with when using this virtual machine. I then proceeded to install Windows on my virtual disk or .vdmk file VMware Workstation created.



I then created an administrator account to access the Windows Server.



The image shows the 'Customize settings' window in Windows Server. It has a blue background with white text. At the top, it says 'Customize settings'. Below that, it says 'Type a password for the built-in administrator account that you can use to sign in to this computer.' There are three input fields: 'User name' with 'Administrator' entered, 'Password' (empty), and 'Reenter password' (empty). At the bottom left is a refresh icon, and at the bottom right is a blue 'Finish' button.

Customize settings

Type a password for the built-in administrator account that you can use to sign in to this computer.

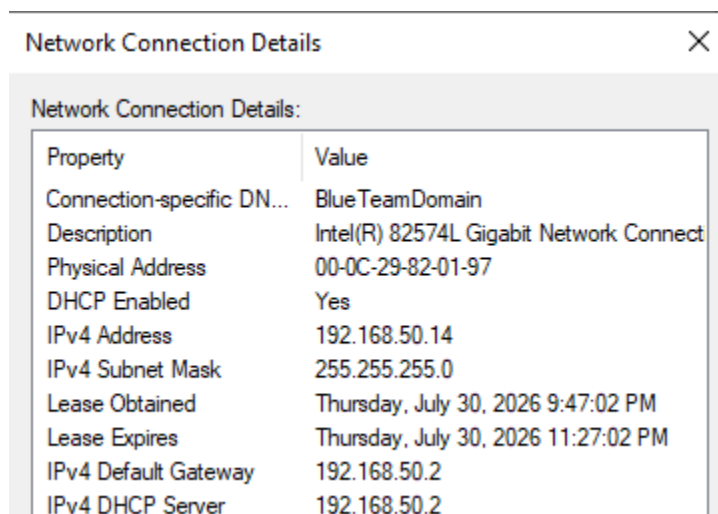
User name Administrator

Password

Reenter password

Finish

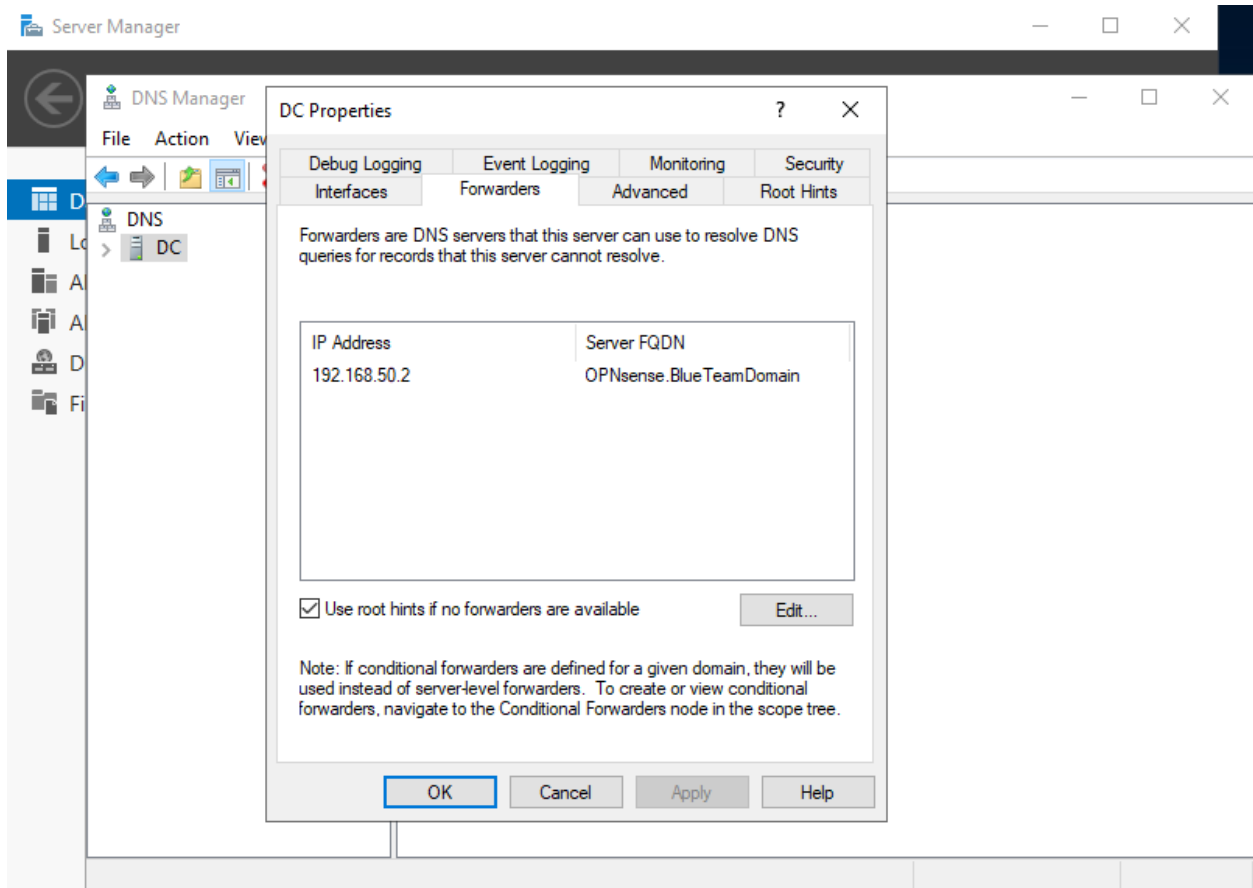
Once the administrator account was created I then looked to verify the IPv4 address resides within the designated LAN subnet and the DNS servers were properly configured.



The image shows the 'Network Connection Details' window. It has a title bar with the text 'Network Connection Details' and a close button. Below the title bar, it says 'Network Connection Details:'. There is a table with two columns: 'Property' and 'Value'.

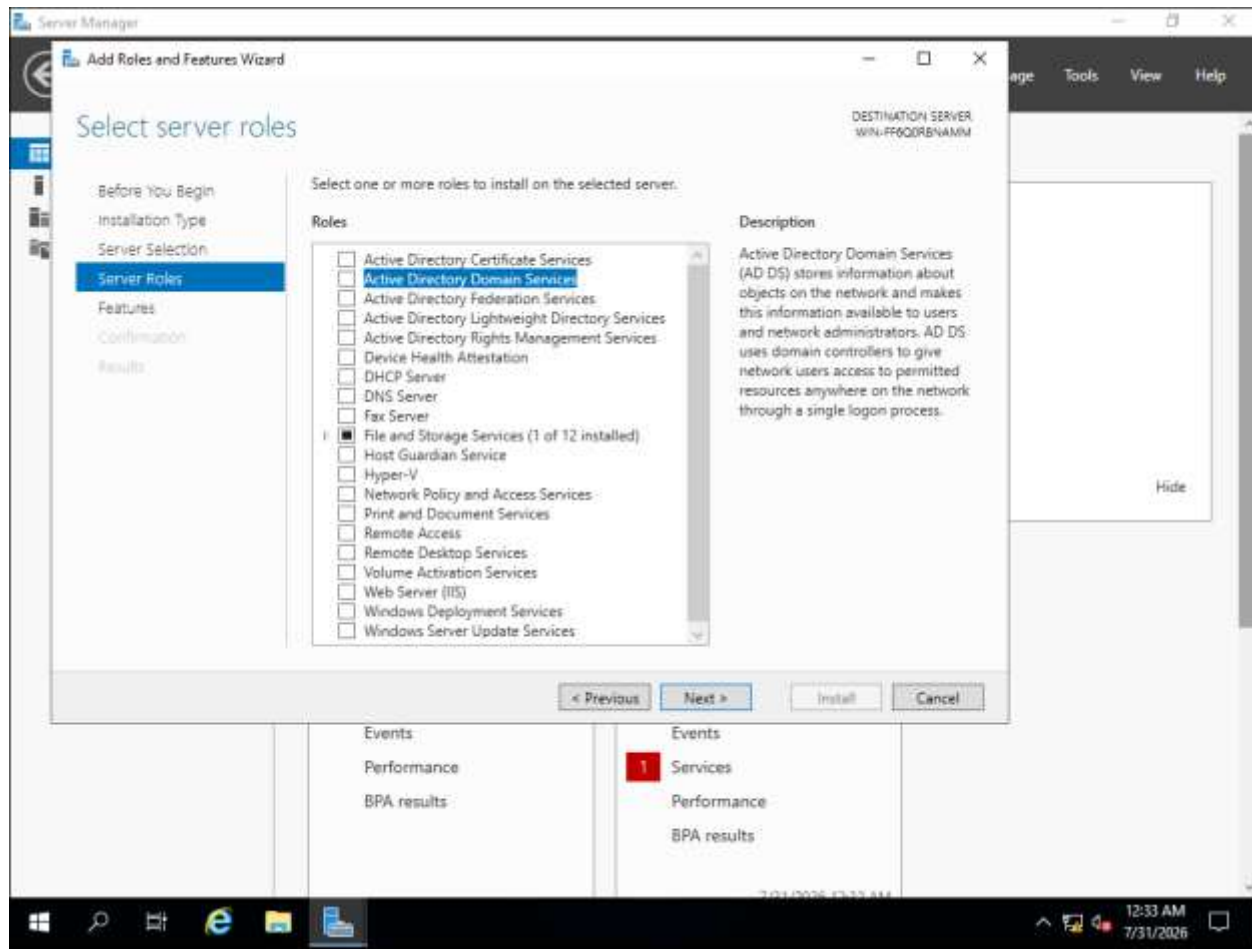
Property	Value
Connection-specific DN...	BlueTeamDomain
Description	Intel(R) 82574L Gigabit Network Connect
Physical Address	00-0C-29-82-01-97
DHCP Enabled	Yes
IPv4 Address	192.168.50.14
IPv4 Subnet Mask	255.255.255.0
Lease Obtained	Thursday, July 30, 2026 9:47:02 PM
Lease Expires	Thursday, July 30, 2026 11:27:02 PM
IPv4 Default Gateway	192.168.50.2
IPv4 DHCP Server	192.168.50.2

Using the Server Manager application, there is a tool named DNS where the administrator can configure the settings made. I opened DNS from the tools tab to ensure the domain controller was using the LAN interface as a forwarder.

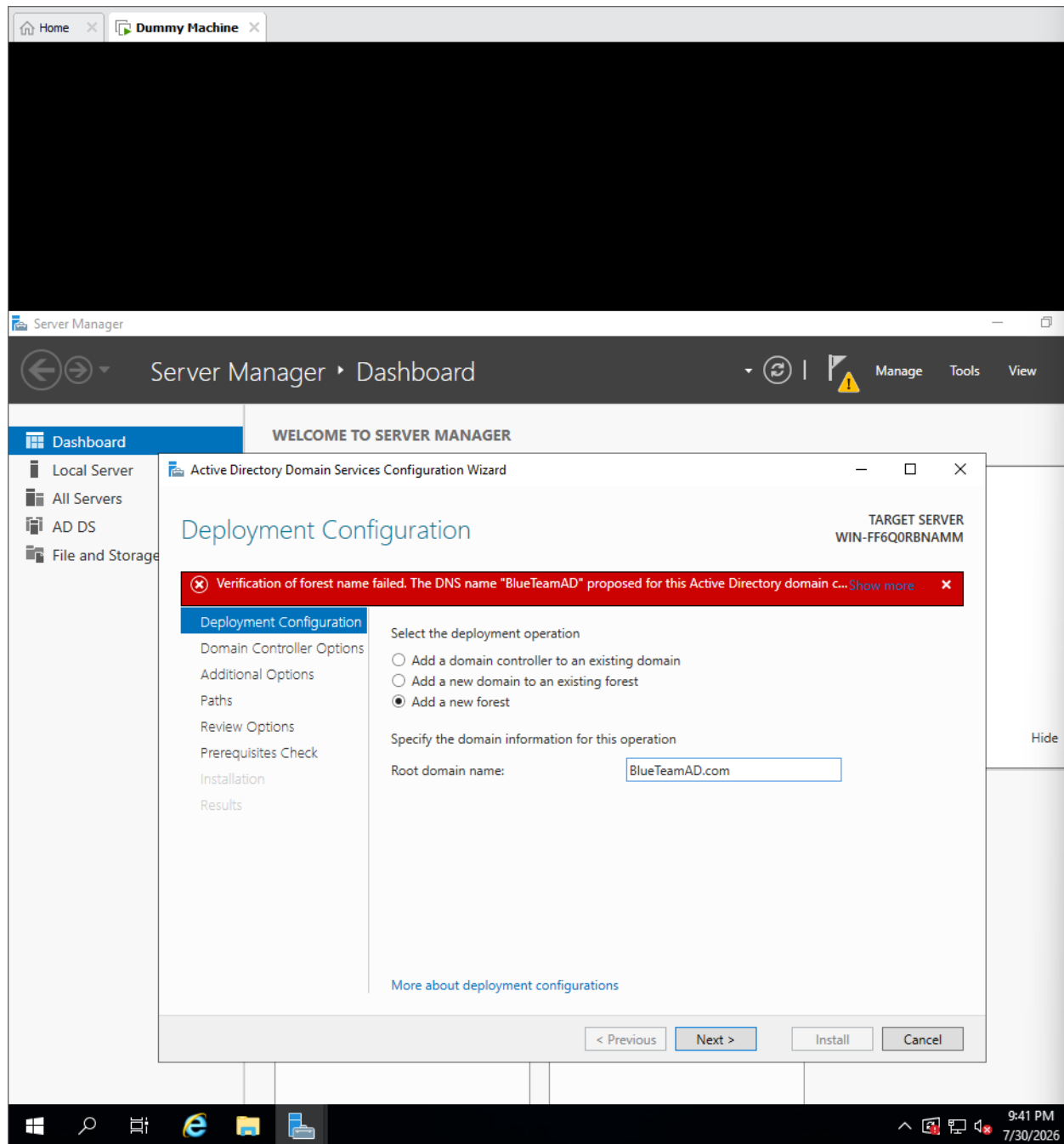


***Note** – The two images above were taken after the domain was created and the server was promoted to domain controller.

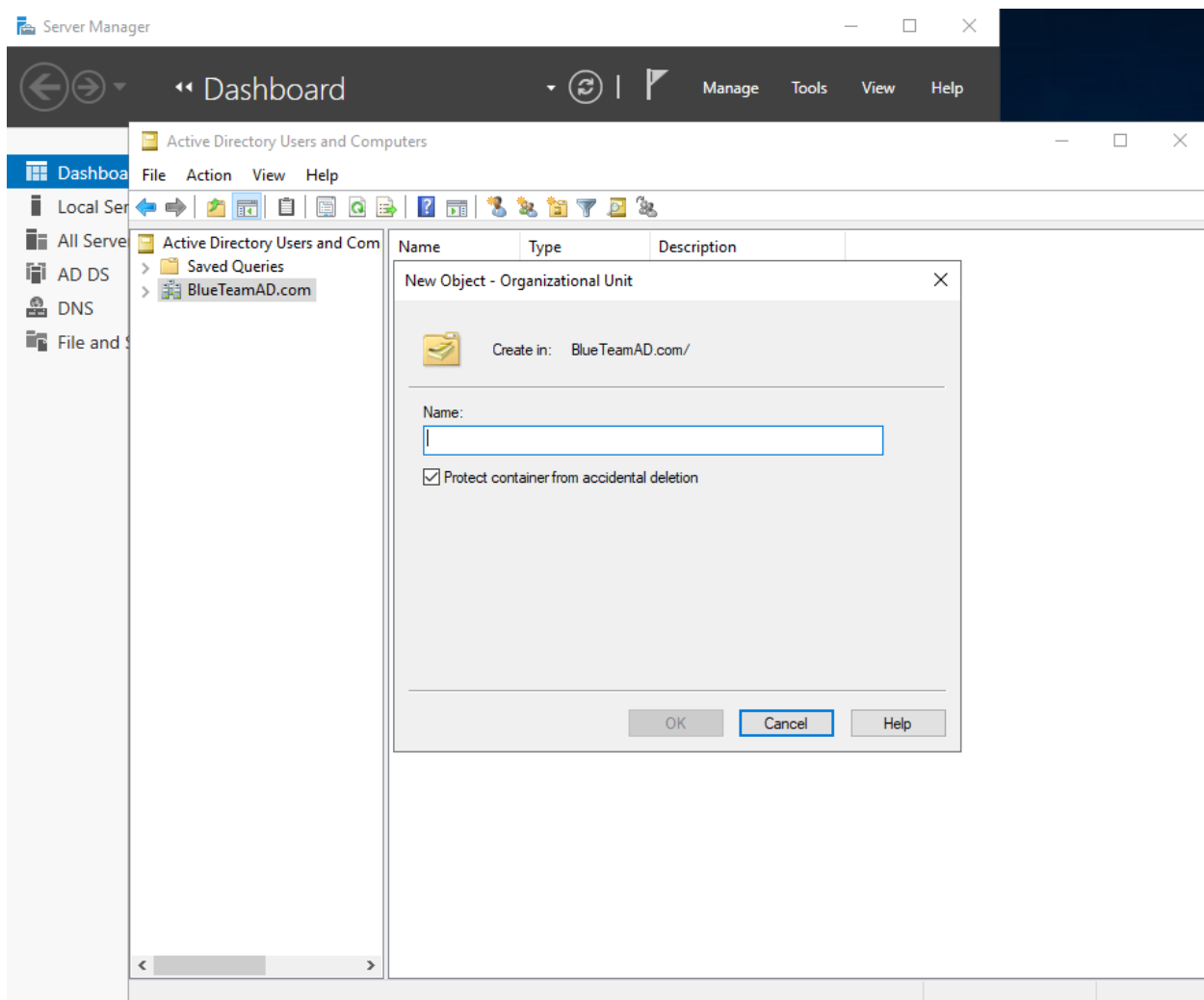
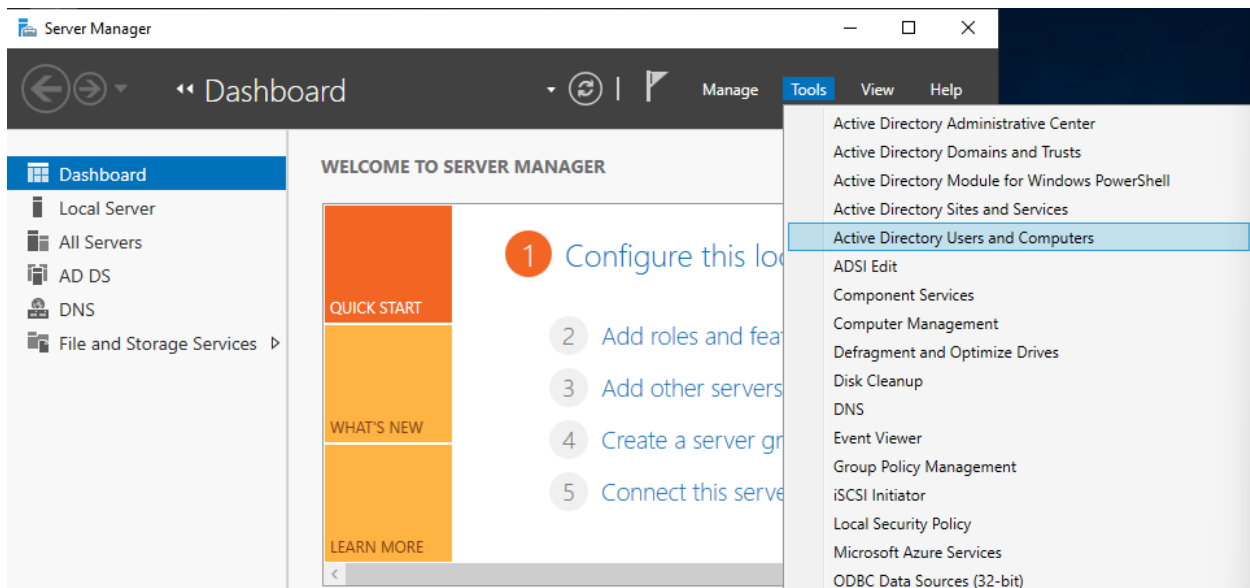
After gaining access to the Windows Server, I then proceeded to the Server Manager application to install Active Directory Domain Services. The reason behind this is I want to create a lab environment that emulates a real enterprise environment where identity access and management is used. I will be downloading this role for the server to practice AD concepts as well as purple team concepts.



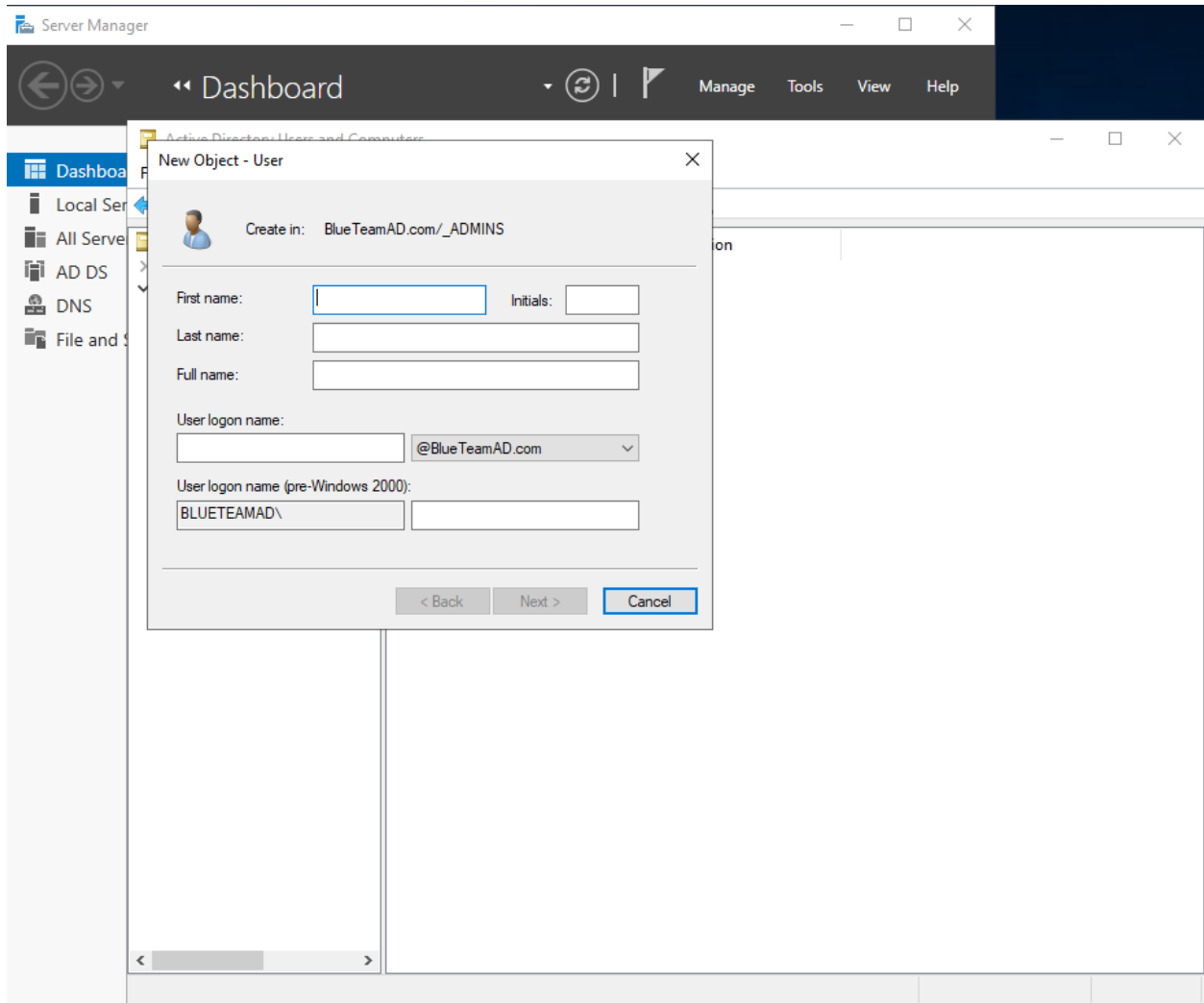
I then proceed to promote my server to domain controller. This set up process prompted me to specify the Root domain name of my server. The domain will be called: BlueTeamAD.



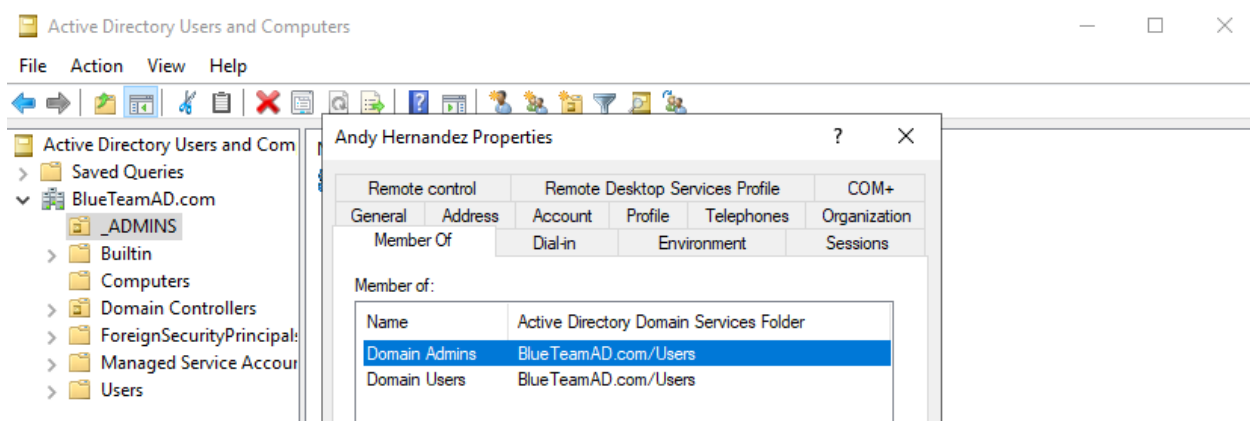
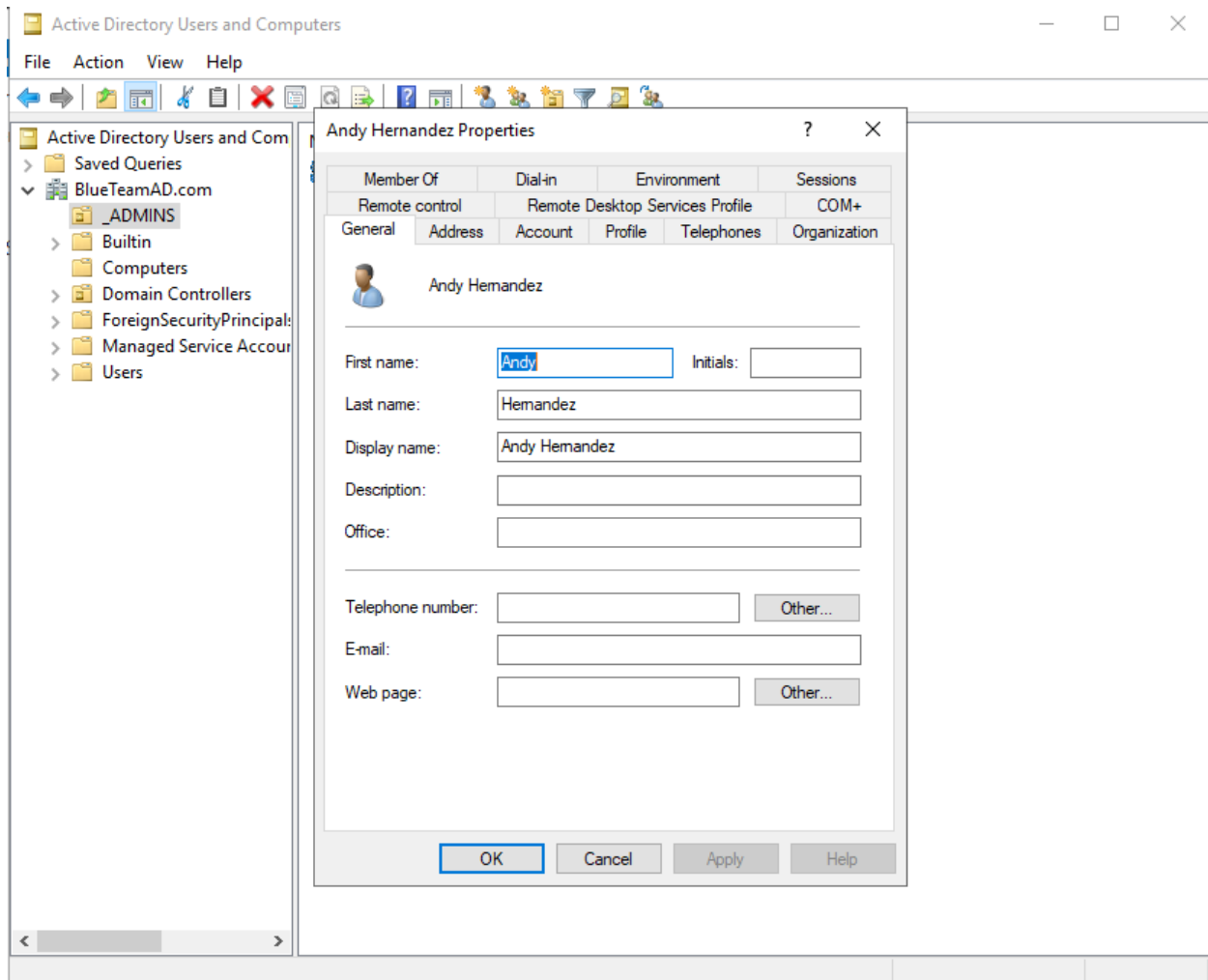
After successfully promoting the Windows Server to domain controller it is now time to create a user that belongs to the group domain administrators. This is the account I will be using to log into the Windows Workstation to validate that the domain was successfully created. I will first begin by accessing tools from the *server manager application* and selecting *Active Directory Users and Computers*. I will then select my domain and create a new *organizational unit*



The new organizational unit will be named _ADMINS. I will then select the new organizational unit and create a new user.



The user will be named Andy Hernandez and will have the username *a-ahernandez*. The first 'a' represents administrator and the second portion represents first and last name. Now that the new user is created I will move forward by adding the new user to be a member of Domain Admins.



Now that most of the configurations have been completed it is important to note that for this domain controller and workstation to work properly two requirements must be met.

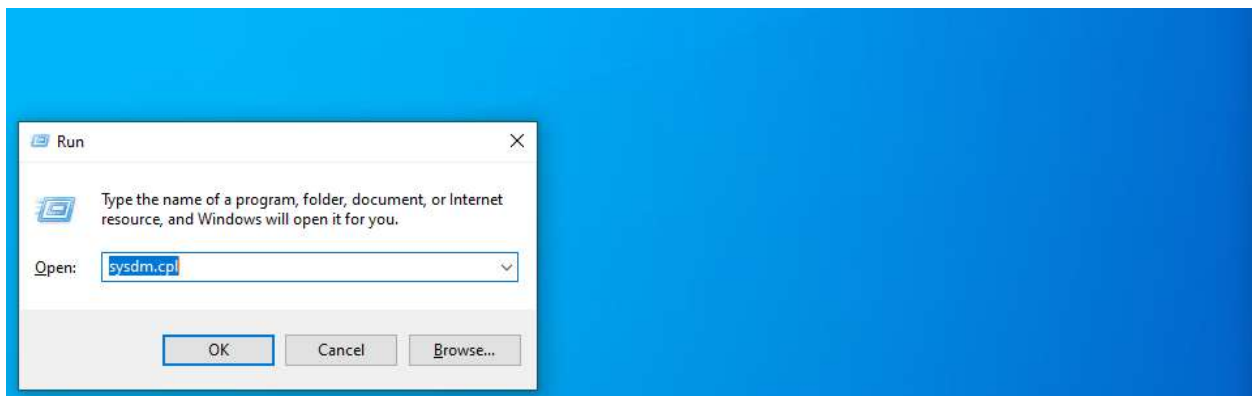
first the domain controller must point to itself as the primary / preferred DNS server and the OPNsense Firewall LAN interface will be the DNS server to forward queries to if unknown by the domain controller.

Second the workstation that will be deployed soon must use the domain controller as the primary / preferred DNS server to join the domain without issue.

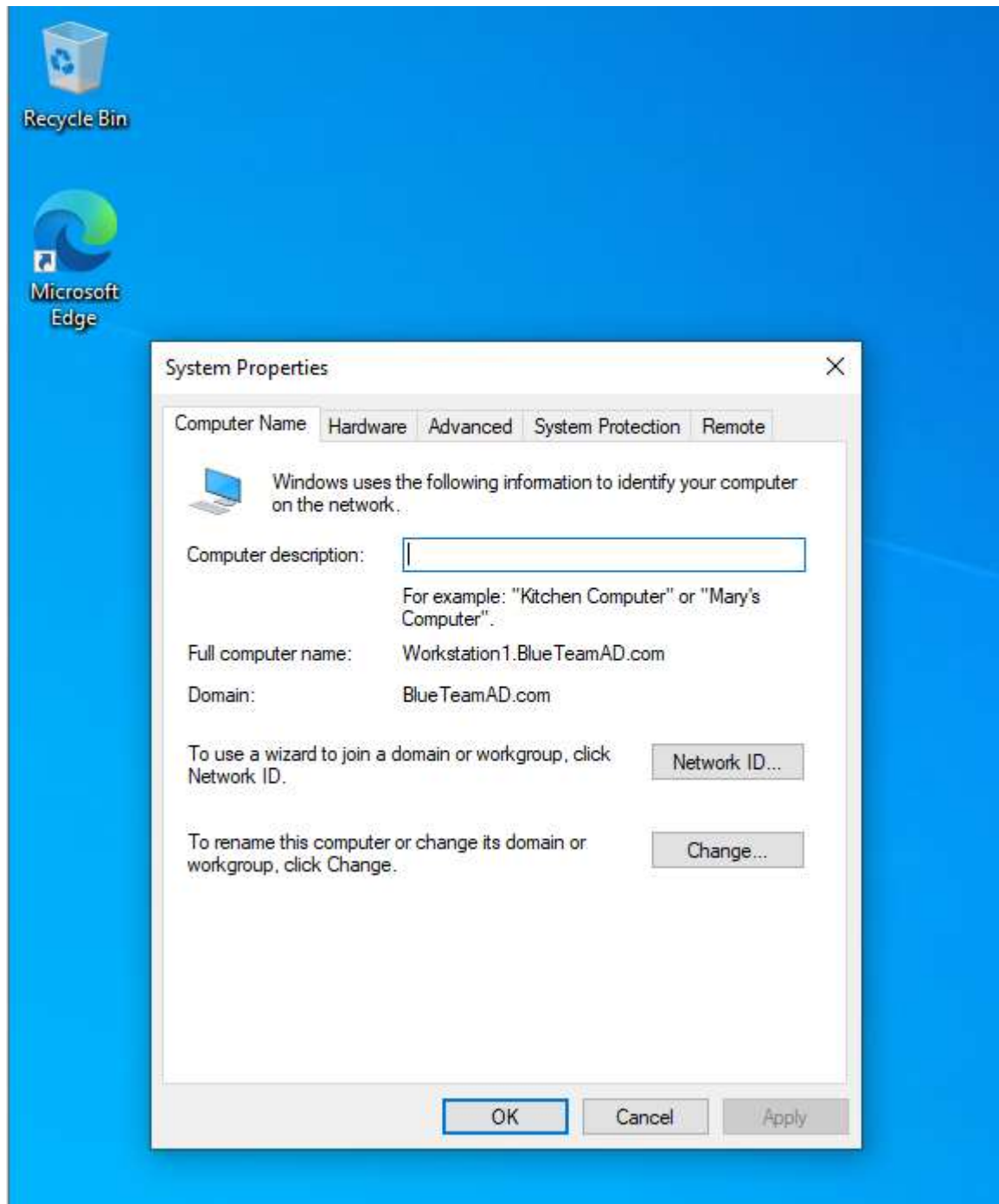
Moving forward I will now deploy the Windows 10 workstation. The installation process is very straightforward, and I will not be covering it in detail. I will only be showing the portion where I add the Workstation to the BlueTeamAD.com domain.

After deploying the Windows 10 workstation and installing the ISO on my virtual disk I proceeded to create a local user account. Using this local account, I will add this workstation to my BlueTeamAD.com domain.

I opened the windows run box and entered “*sysdm.cpl*” to open a systems properties box.

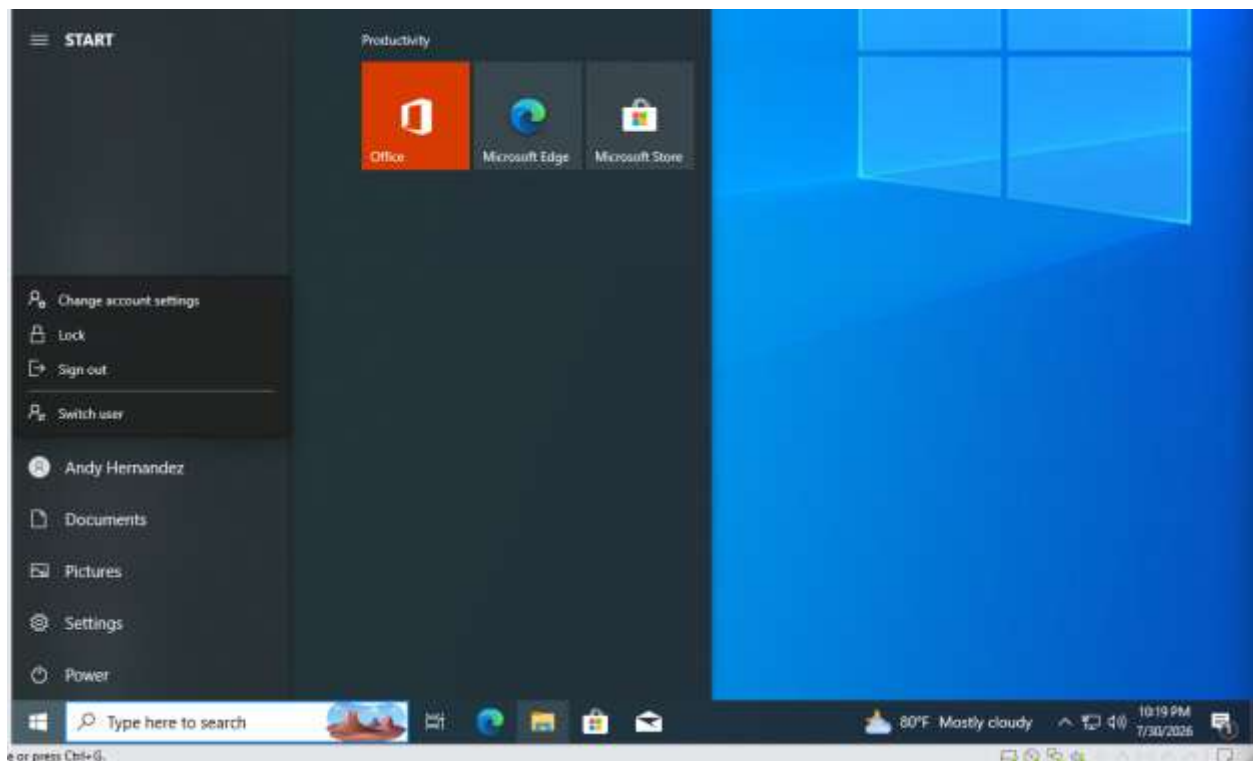
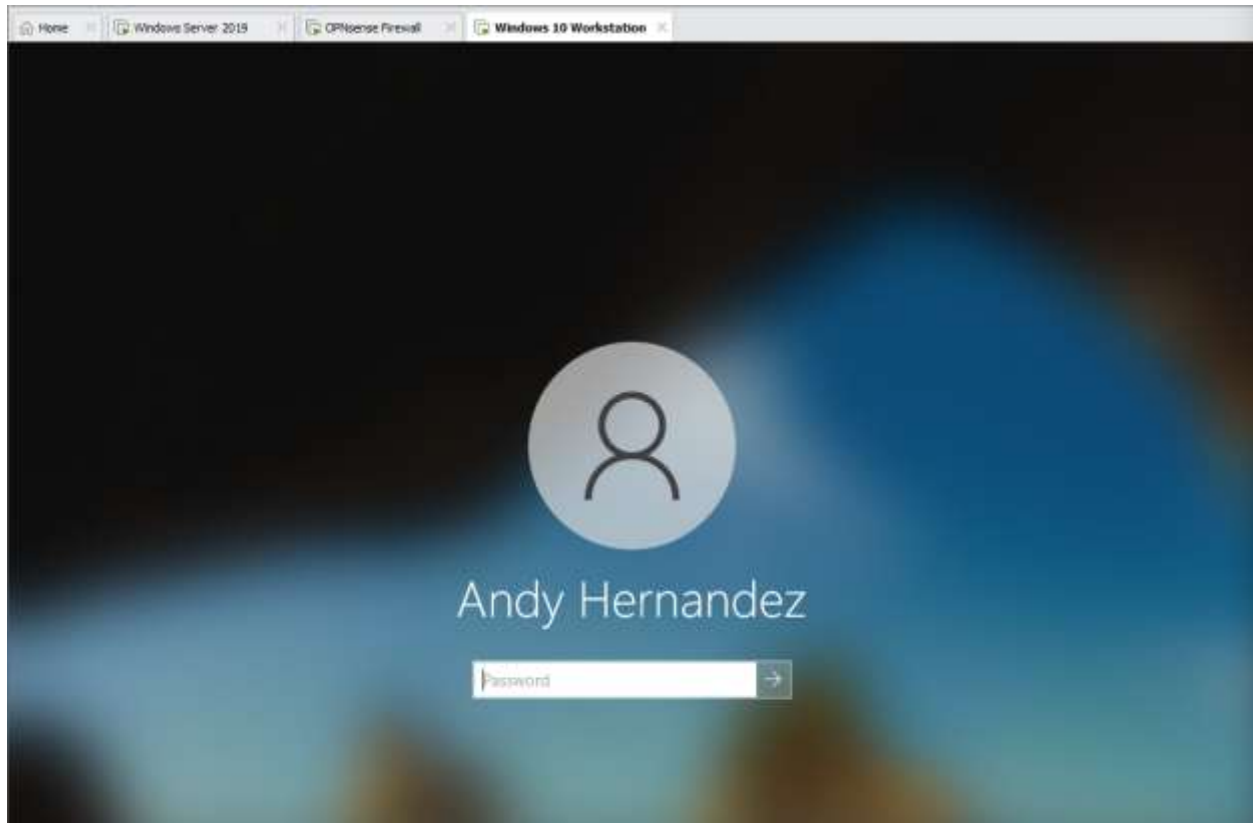


From here I renamed the computer to **Workstation1** and added the new system to the domain after successfully authenticating the new system with the administrator credentials.



I then logged out and proceeded to login with the new user account I created.

I was able to successfully log into Workstation1.BlueTeamAD.com with the account belonging to Andy Hernandez or in other words *a-ahernandez*.



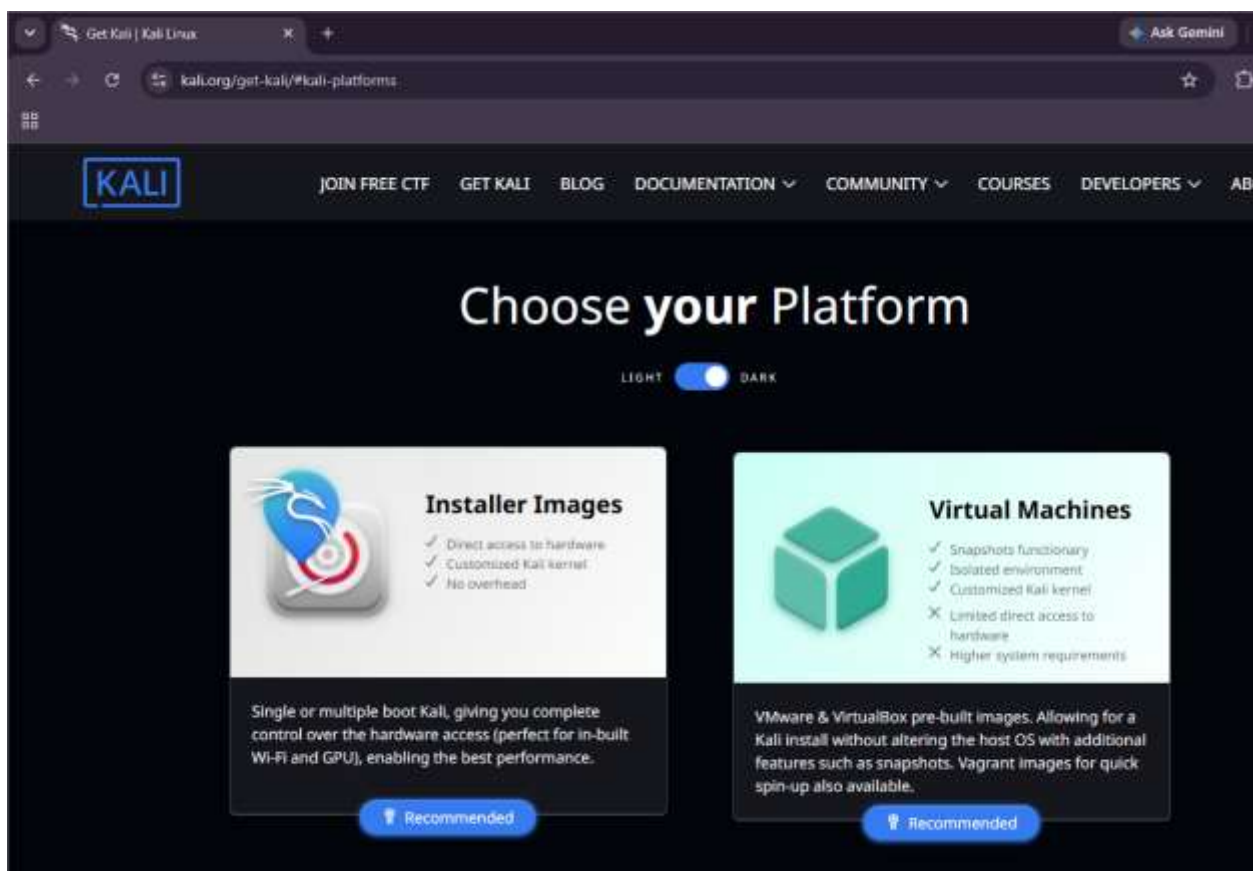
With both the Windows Server and Windows 10 Workstation successfully configured, it was time to deploy the Kali Linux Virtual Machine using the virtual network interface VMnet8.

Kali Linux: This portion of the project is very straightforward as all you need to do is download the Kali Linux ISO and deploy it using VMware Workstation. VMnet8 is configured to use DHCP so it will be a very simple setup.

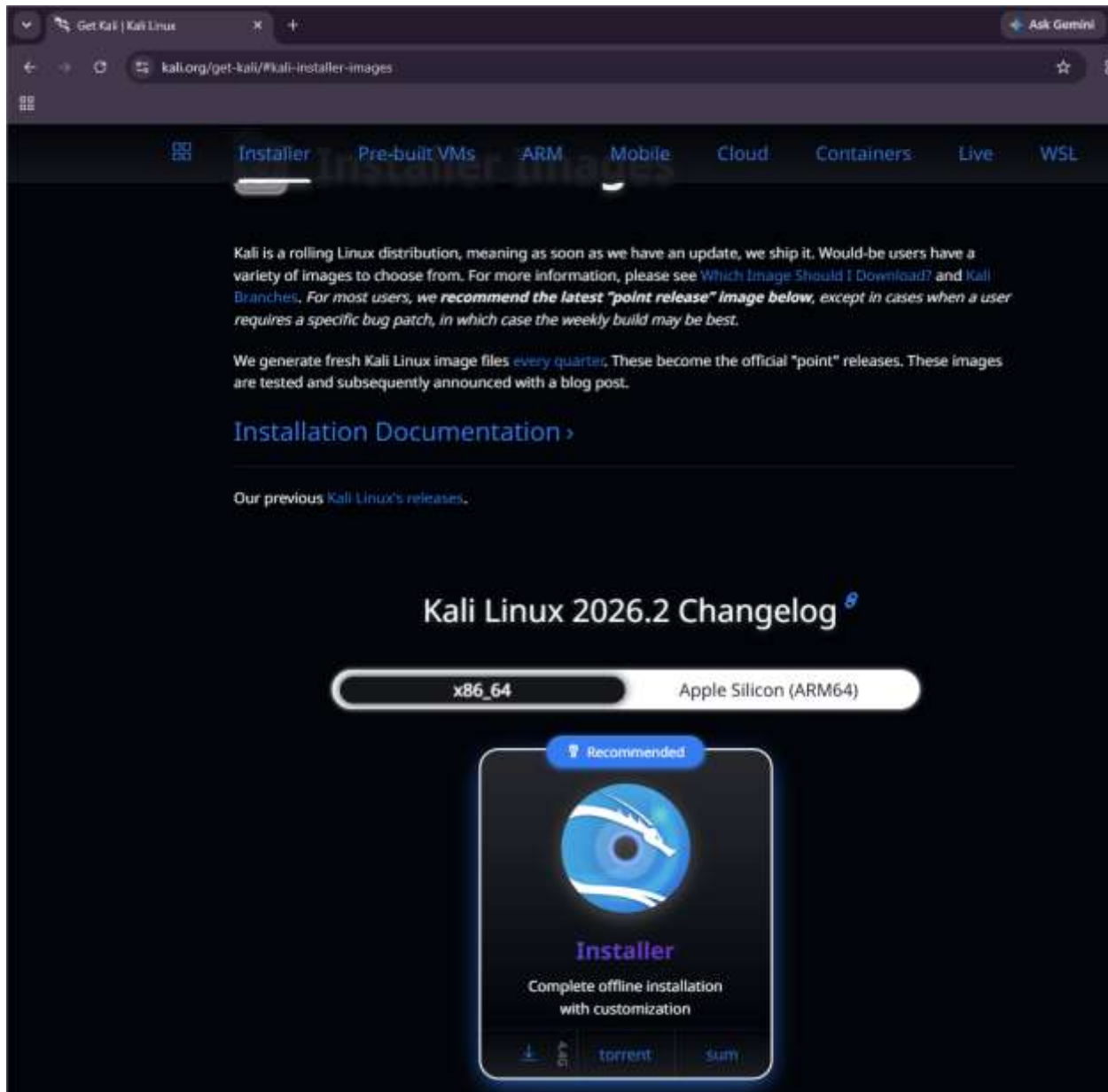
First I will head to this URL to download the Kali Linux ISO:

<https://www.kali.org/get-kali/#kali-platforms>

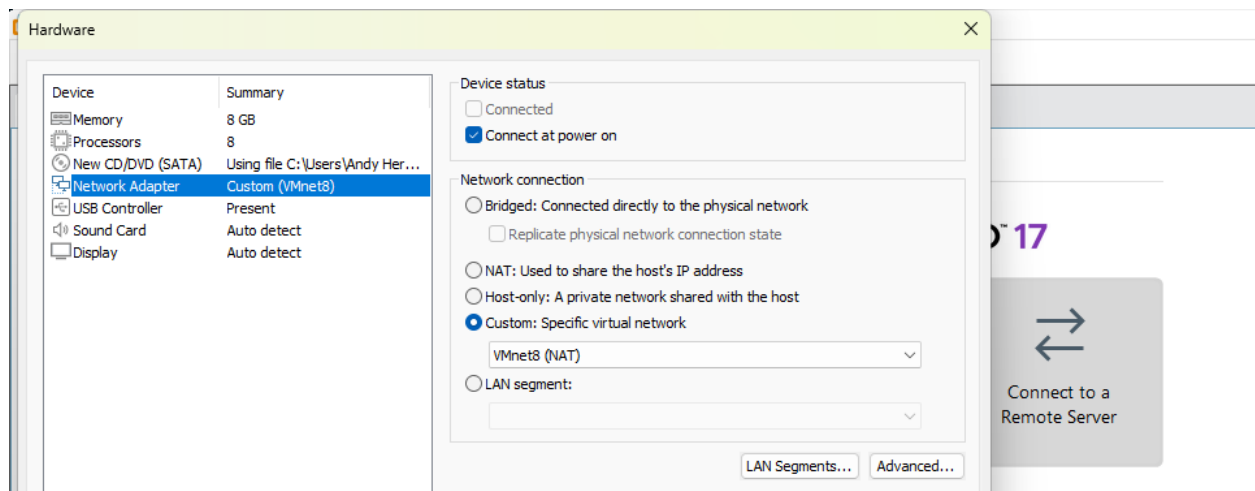
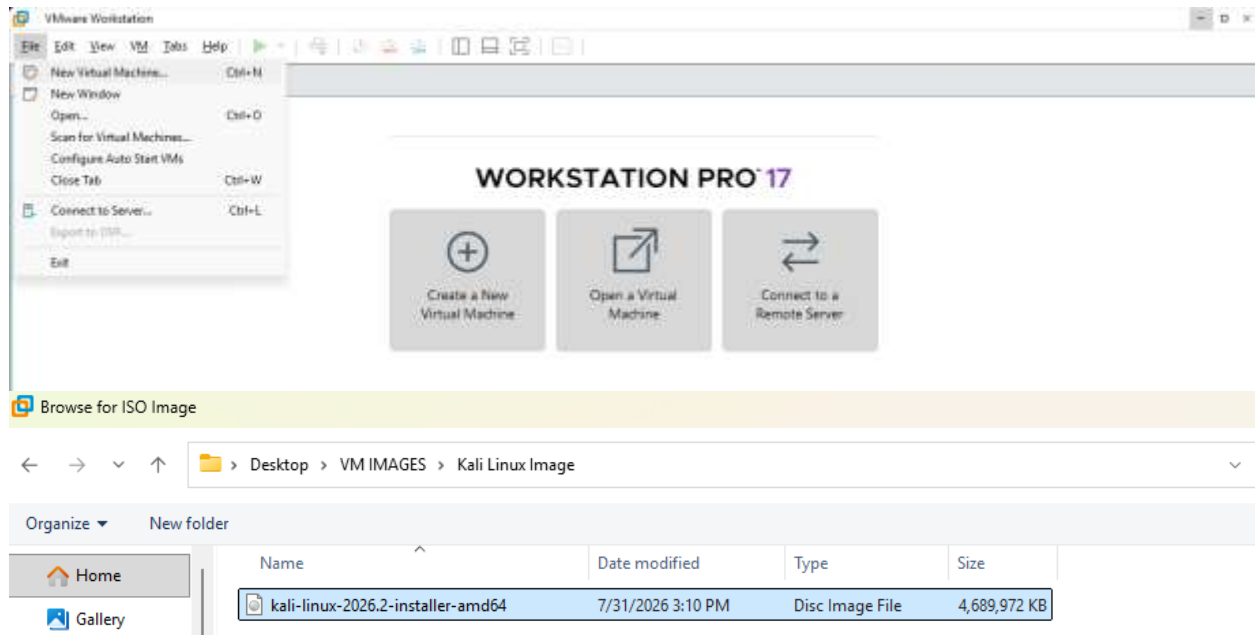
I will then click on the installer images as I want a .ISO Image I can deploy on VMware Workstation.

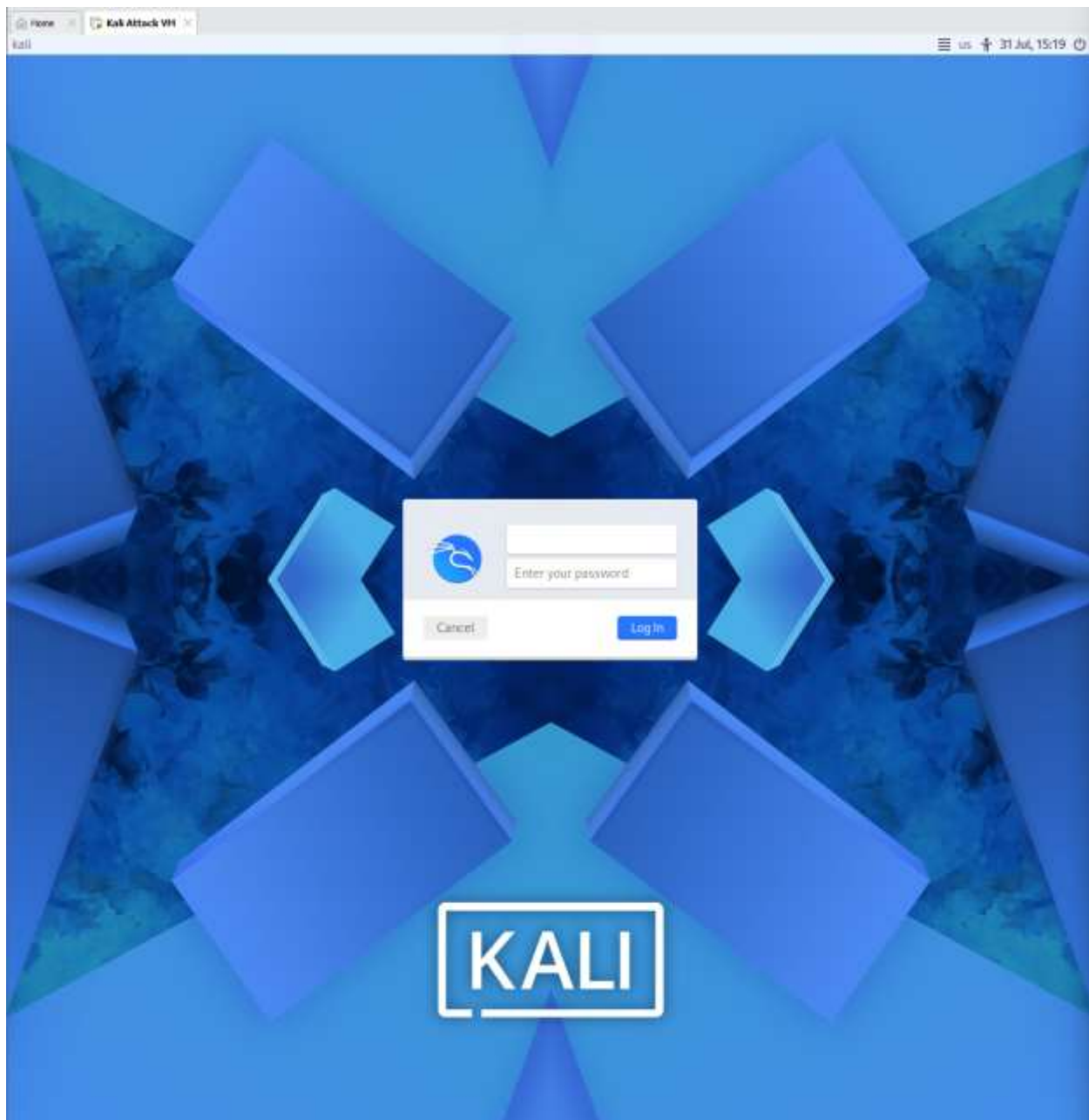


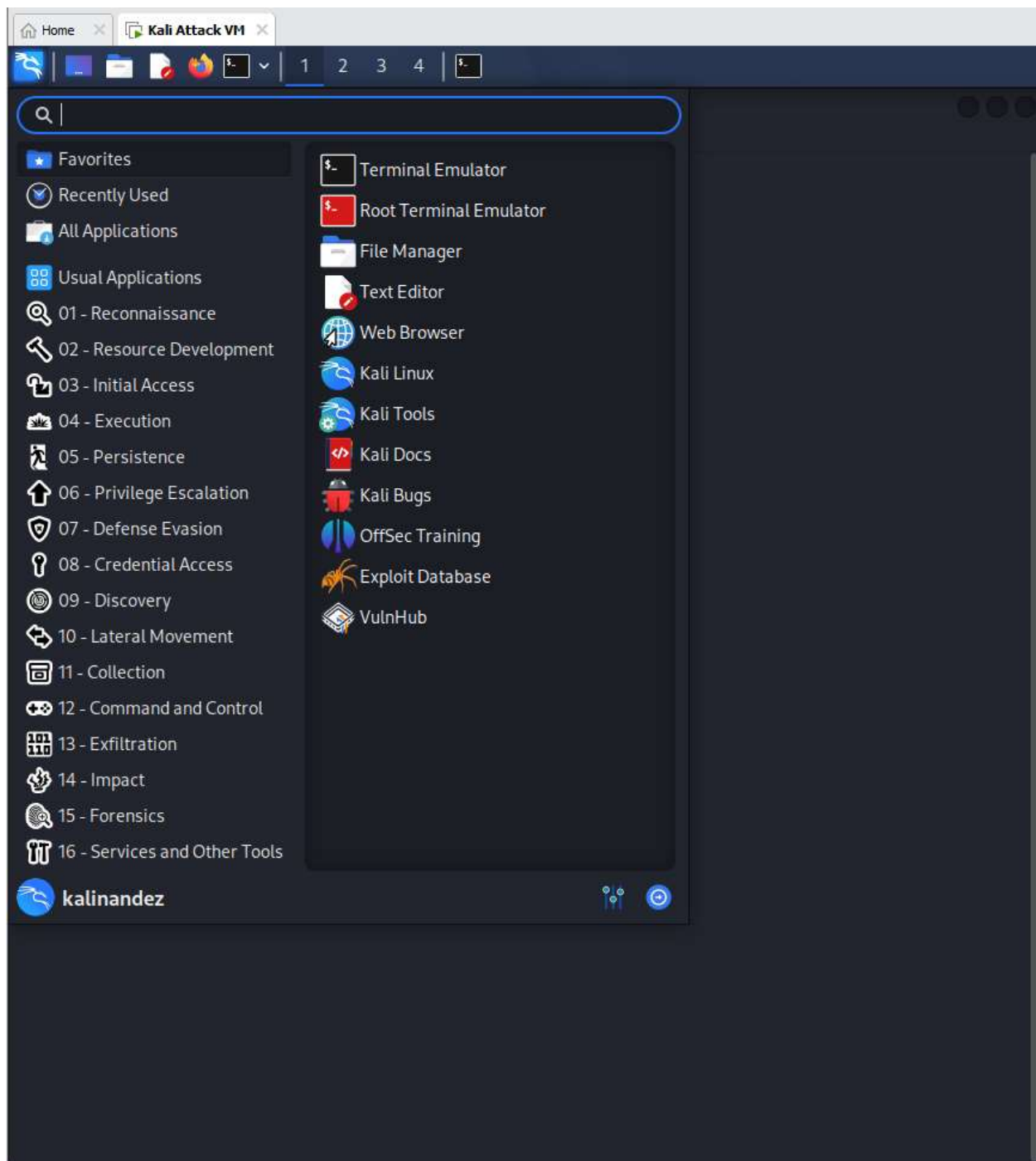
I will download the recommended Installer Image as seen below



Once done I will configure the allocated resources, virtual network adapter (VMnet8), follow the installation instructions and create user account to access the virtual machine.







After initial deployment I always check to see if the virtual machine has the appropriate IPv4 address or in other words has an IPv4 address appropriate to its subnet. After that I check to see if the virtual machine has a stable internet connection and update the preinstalled packages.

```
kalinandez@kali: ~  
Session Actions Edit View Help  
(kalinandez@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:43:05:dc brd ff:ff:ff:ff:ff:ff  
    inet 192.168.116.130/24 brd 192.168.116.255 scope global dynamic noprefixroute eth0  
        valid_lft 1434sec preferred_lft 1434sec  
    inet6 fe80::20c:29ff:fe43:5dc/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(kalinandez@kali)-[~]  
$
```

```
kalinandez@kali: ~  
Session Actions Edit View Help  
(kalinandez@kali)-[~]  
$ ping google.com  
PING google.com (142.251.35.238) 56(84) bytes of data:  
64 bytes from tzmiaa-aa-in-f14.1e100.net (142.251.35.238): icmp_seq=1 ttl=128 time=16.1 ms  
64 bytes from tzmiaa-aa-in-f14.1e100.net (142.251.35.238): icmp_seq=2 ttl=128 time=13.1 ms  
64 bytes from tzmiaa-aa-in-f14.1e100.net (142.251.35.238): icmp_seq=3 ttl=128 time=12.5 ms  
64 bytes from tzmiaa-aa-in-f14.1e100.net (142.251.35.238): icmp_seq=4 ttl=128 time=15.9 ms  
^C  
--- google.com ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3006ms  
rtt min/avg/max/mdev = 12.480/14.421/16.141/1.630 ms  
(kalinandez@kali)-[~]  
$
```

```
root@kali: ~  
Session Actions Edit View Help  
(root@kali)-[~]  
$ apt upgrade -y  
The following packages were automatically installed and are no longer required:  
curlftpfs libfuse2t64 libpocketsphinx3 libteandctl0 python3-click-plugins  
enchant-2 libgav1-1 libpoppler147 libtsk19t64 python3-fs  
firmware-marvell-presters libgdal37 libpostali libunlbreak6 python3-multipart  
git1.2-girepository-2.0 libgdal38 libpostproc58 libvdpau-va-gll python3-pluginbase  
libarmadillo14 libgdk-pixbuf2.0-bin libradare2-5.0.0t64 libvidstab1.1 python3-pysmi
```

```
uto mode  
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/lib/jexec to provide /usr/bin/jexec (jexec) in auto mode  
Setting up libpango-1.0-0:amd64 (1.58.0-1)..  
Setting up libsrtp2-1:amd64 (2.8.0+ds-1)..  
Setting up python3-typeguard (4.4.4-1)..  
Setting up libdrm-intel1:amd64 (2.4.134-1)..  
Setting up libibus-1.0-5:amd64 (1.5.34-1)..  
Setting up libraptor2-0:amd64 (2.0.16-7)..  
Setting up php8.4-common (8.4.23-1)..  
Progress: [ 81% ]
```

Now that the deployment of the Kai Linux machine is finished, it is time to conclude the phase one of this SOC home lab.

Overall, the first phase of the project was very insightful as I learned more about basic troubleshooting and networking concepts while creating the network architecture and setting up the lab environment. Recapping on everything learned throughout this phase:

- Basic troubleshooting
- Networking concepts
- Configuring and troubleshooting network interfaces
- Configuring and troubleshooting DHCP & DNS
- Creating and tuning firewall rules
- Creating an Active Directory Domain and Users
- Joining Workstations to the Active Directory Domain
- Deploying and troubleshooting virtual machines within VMware Workstation
- Adding repositories
- Updating packages using dnf or apt.

This concludes the setup of the lab environment and phase one of this project.